同济大学 TONGJI UNIVERSITY

同济大学 TONGJI UNIVERSITY

图像分析与机器学习

2 线性分类与感知机



张绍明



目录

1. 线性判别函数
   - ✓ 最小二乘求解
   - ✓ Fisher准则求解（LDA）
2. 感知机
3. 类别后验分布与线性判别模型
4. Logistic回归
   - ✓ Logistic回归与感知机算法的对比
   - ✓ Logistic回归与线性回归的对比

分类判别函数

- 两类问题
  - 线性判别函数
  - 几何意义
- K类问题
  - 目标向量的表达（对比回归问题）
  - One V.S. The Rest
  - One V.S. One
  - K类线性判别函数

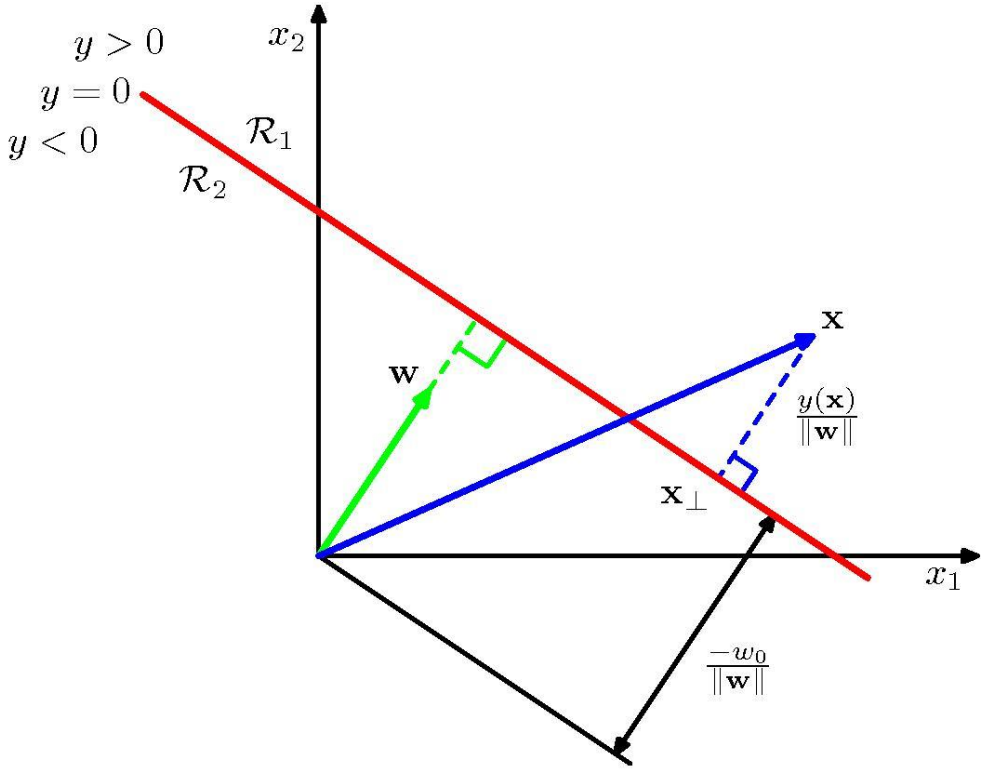
分类判别函数

- 判别函数把自变量x映射到K个类别中的一个C_k，我们从K=2的情况开始展开分析。
- 如果判别函数的形式为线性，可描述如下：

$$y = \mathbf{w}^T\boldsymbol{\phi}(\mathbf{x}) + w_0 \text{ 或 } y = \mathbf{w}^T\boldsymbol{\phi}(\mathbf{x}) + b$$

- 注意：线性是指对基函数（特征向量）
- $y = \mathbf{w}^T\boldsymbol{\phi}(\mathbf{x}) + w_0 = 0$是决策边界

- $\mathbf{x} \in \begin{cases} C_1, if\ y \geq 0 \\ C_2, if\ y < 0 \end{cases}$



分类判别函数（几何意义）

暂考虑最简单情况，$\boldsymbol{\phi}(\mathbf{x}) = \mathbf{x}$

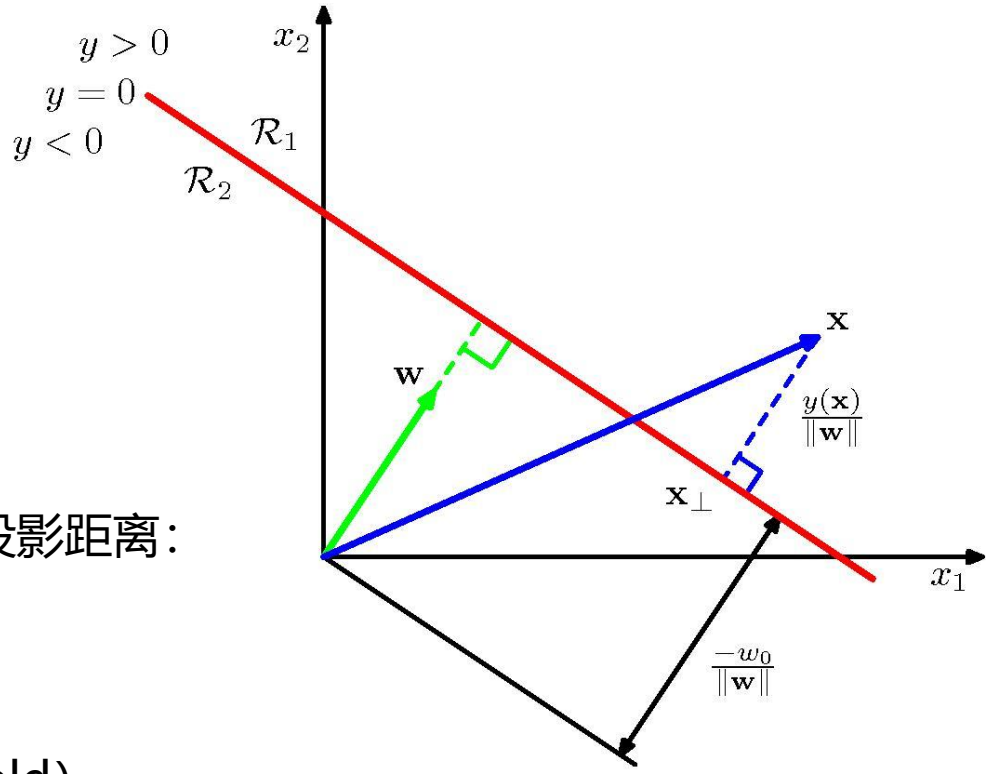
- 决策边界上的两个点$\mathbf{x}_a$和$\mathbf{x}_b$
- 由$y = \mathbf{w}^T\mathbf{x} + w_0 = 0$可知：

$$\mathbf{w}^T\mathbf{x}_a + w_0 = \mathbf{w}^T\mathbf{x}_b + w_0 = 0，\text{即}\mathbf{w}^T(\mathbf{x}_a - \mathbf{x}_b) = 0$$

- 判别函数系数向量与决策边界正交
- 原点到决策边界的距离：决策边界上任意点$\mathbf{x}_d$在$\mathbf{w}$方向的投影距离：

$$\left\langle \frac{\mathbf{w}}{\|\mathbf{w}\|}, \mathbf{x}_d \right\rangle = \frac{\mathbf{w}^T\mathbf{x}_d}{\|\mathbf{w}\|} = \frac{\|\mathbf{w}\|\|\mathbf{x}_d\|cos\theta}{\|\mathbf{w}\|} = \frac{-w_0}{\|\mathbf{w}\|}$$

可知决策边界的位置由w_0决定，$-w_0$也被称为阈值（Threshold）



分类判别函数（几何意义）

- 原点到决策边界的距离：决策边界上任意点$\mathbf{x}_d$在$\mathbf{w}$方向的投影距离：

$$\left\langle \frac{\mathbf{w}}{\|\mathbf{w}\|}, \mathbf{x}_d \right\rangle = \frac{\mathbf{w}^T\mathbf{x}_d}{\|\mathbf{w}\|} = \frac{\|\mathbf{w}\|\|\mathbf{x}_d\|cos\theta}{\|\mathbf{w}\|} = \frac{-w_0}{\|\mathbf{w}\|}$$

- $\mathbf{x}$到决策边界的垂足为$\mathbf{x}_\perp$，距离为r

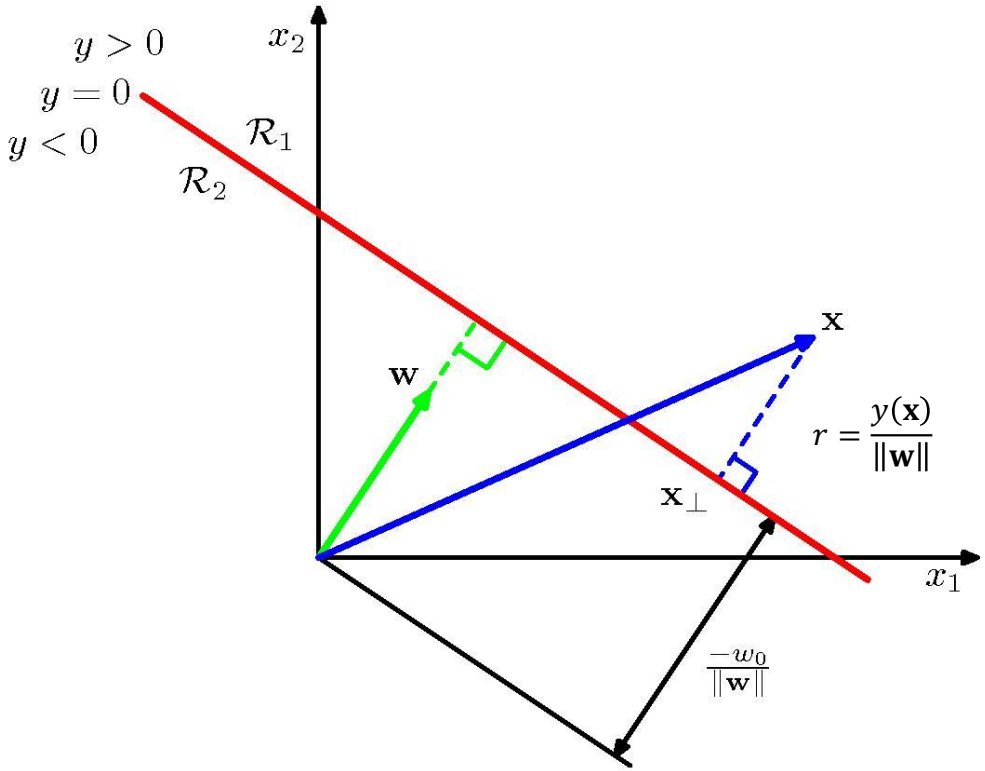
$$\mathbf{x} = \mathbf{x}_\perp + r\frac{\mathbf{w}}{\|\mathbf{w}\|}$$

$$\mathbf{w}^T\mathbf{x} + w_0 = \mathbf{w}^T\mathbf{x}_\perp + w_0 + r\frac{\mathbf{w}^T\mathbf{w}}{\|\mathbf{w}\|}$$

决策边界上的点y=0

$$y(\mathbf{x}) = r\frac{\|\mathbf{w}\|^2}{\|\mathbf{w}\|} \rightarrow r = \frac{y(\mathbf{x})}{\|\mathbf{w}\|}$$

- 判别函数的值体现的是数据到决策边界的距离r



分类判别函数（K类问题）

1. 目标向量的表达（对比回归问题）
2. 用多个两类判别函数表达多类问题
   - One V.S. The Rest
   - One V.S. One
3. K个线性判别函数表达K类问题

分类判别函数（K类问题）

1. 对于K类问题，类别（目标值、y值）的表达不再使用标量
2. 使用1-of-K 编码方式的向量
3. 例如K = 5, y值分别为：
   1. $(1,0,0,0,0)^T$
   2. $(0,1,0,0,0)^T$
   3. $(0,0,1,0,0)^T$
   4. $(0,0,0,1,0)^T$
   5. $(0,0,0,0,1)^T$

分类判别函数（K类问题）

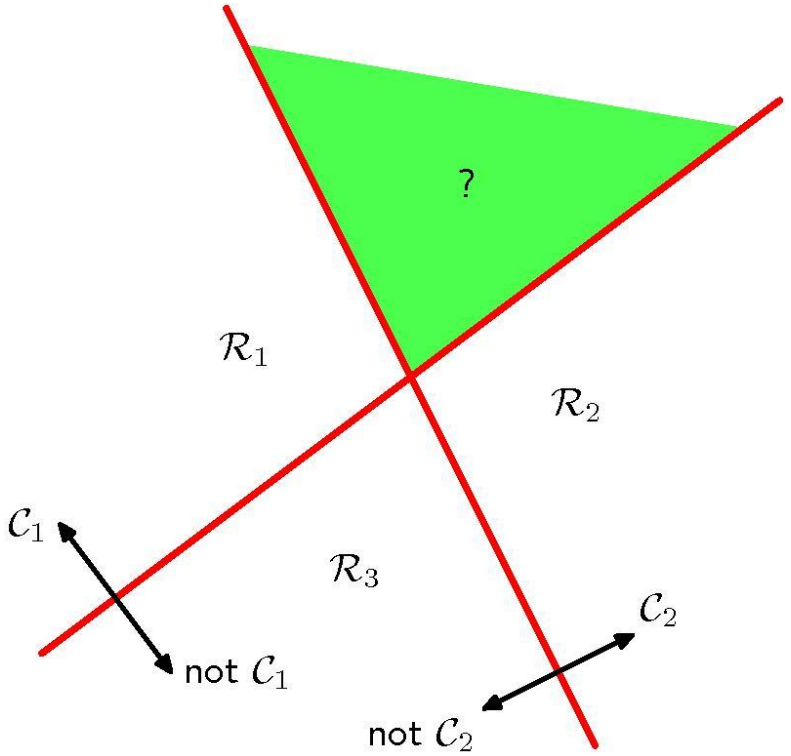
K类问题

1. 用多个两类判别函数表达多类问题
   - One V.S. The Rest
   - One V.S. One
2. K个线性判别函数表达K类问题

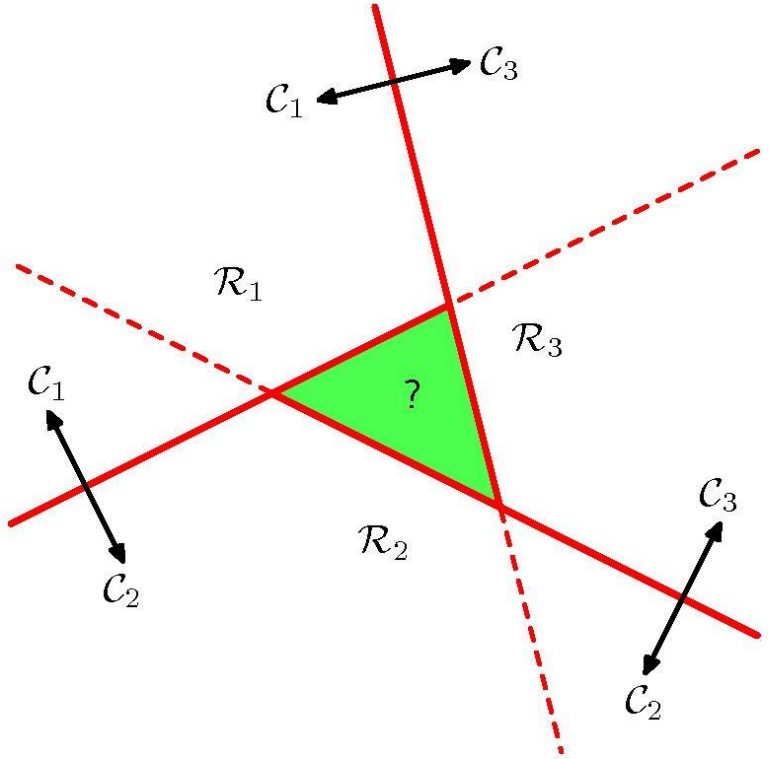
分类判别函数（K类问题）

用多个两类判别函数表达多类问题，存在类别模糊区域

One V.S. The Rest　K-1个



One V.S. One　K(K-1)/2 个



分类判别函数（K类问题）

- K个线性判别函数表达K类问题，暂只考虑$\boldsymbol{\phi}(\mathbf{x}) = \mathbf{x}$

$$y_k(\mathbf{x}) = \mathbf{w}_k^T \boldsymbol{\phi}(\mathbf{x}) + w_{k0}$$

$$\mathbf{x} \in C_k, if\ y_k(\mathbf{x}) \geq y_j(\mathbf{x}), k \neq j$$

- C_k和C_j之间的决策边界由$y_k(\mathbf{x}) = y_j(\mathbf{x})$确定，如果特征维度为M，决策边界是一个M-1维的超平面：$\left(\mathbf{w}_k^T - \mathbf{w}_j^T\right)\boldsymbol{\phi}(\mathbf{x}) + \left(w_{k0} - w_{j0}\right) = 0$
- 上式与二类问题的决策边界$\mathbf{w}^T \boldsymbol{\phi}(\mathbf{x}) + w_0 = 0$表达形式相同。

分类判别函数（K类问题）

- K个线性判别函数表达K类问题

$$y_k(\mathbf{x}) = \mathbf{w}_k^T \boldsymbol{\phi}(\mathbf{x}) + w_{k0}$$

$$\mathbf{x} \in C_k, if\ y_k(\mathbf{x}) \geq y_j(\mathbf{x}), k \neq j$$

$(\mathbf{0.05}, \mathbf{0.01}, \mathbf{0.92}, \mathbf{0.01}, \mathbf{0.01})^T$

预测为第3类

- 如何通过已经获取的训练数据，获取K组参数？ $\tilde{\mathbf{w}}_k = \left(w_{k0}, \mathbf{w}_k^T\right)^T$

1. 最小二乘
2. Fisher线性判别分析（LDA）
3. 感知机算法（Perceptron）

最小二乘求解判别函数

1. 回顾最小二乘回归问题：$Loss(\mathbf{w}) = \frac{1}{2}\|\mathbf{\Phi w} - \mathbf{t}\|^2 = \frac{1}{2}\sum_{n=1}^{N}(\mathbf{w}^T\boldsymbol{\phi}(x_n) - t_n)^2$

$$\mathbf{\Phi} = \begin{pmatrix} \boldsymbol{\phi}(x_1)^T \\ \boldsymbol{\phi}(x_2)^T \\ \vdots \\ \boldsymbol{\phi}(x_N)^T \end{pmatrix} = \begin{pmatrix} \phi_0(x_1) & \phi_1(x_1) & \cdots & \phi_M(x_1) \\ \phi_0(x_2) & \phi_1(x_2) & \cdots & \phi_M(x_2) \\ \vdots & \vdots & \ddots & \vdots \\ \phi_0(x_N) & \phi_1(x_N) & \cdots & \phi_M(x_N) \end{pmatrix} \qquad \mathbf{t} = \begin{pmatrix} t_1 \\ t_2 \\ \vdots \\ t_N \end{pmatrix}$$

2. 最小二乘线性判别分类问题：目标值 t_n 不再是标量，而是形如 $\mathbf{t}_n = (0,1,0,0,0)^T$ 的1-of-K向量。

3. 在预测时，预测向量中第k个元素y_k由第k组w求得（$y_k(\mathbf{x}) = \tilde{\mathbf{w}}_k^T\boldsymbol{\phi}(\mathbf{x})$）。

4. $\mathbf{\Phi}$的表达式不变，$\mathbf{t}$变为$\mathbf{T} = \begin{pmatrix} \mathbf{t}_1^T \\ \mathbf{t}_2^T \\ \vdots \\ \mathbf{t}_N^T \end{pmatrix}$，**参数由一组**变为K组，即$\mathbf{w}$变为$\mathbf{W} = (\tilde{\mathbf{w}}_1, \tilde{\mathbf{w}}_2, \cdots, \tilde{\mathbf{w}}_K)$，每一列对应一组参数。

最小二乘求解判别函数

- $\mathbf{t}_n = \begin{pmatrix} t_{n1} \\ t_{n2} \\ \vdots \\ t_{nK} \end{pmatrix}$ V.S. $\mathbf{y}_n(\mathbf{x}_n) = \begin{pmatrix} y_{n1} \\ y_{n2} \\ \vdots \\ y_{nK} \end{pmatrix}$, $y_{nk} = \tilde{\mathbf{w}}_k^T \boldsymbol{\phi}(\mathbf{x}_n)$

$$\mathbf{t}_n = \begin{pmatrix} 0 \\ 0 \\ 1 \\ 0 \\ 0 \end{pmatrix} \quad \text{V.S.} \quad \mathbf{y}_n(\mathbf{x}_n) = \begin{pmatrix} 0.05 \\ 0.01 \\ 0.92 \\ 0.01 \\ 0.01 \end{pmatrix} = \begin{pmatrix} \tilde{\mathbf{w}}_0^T \boldsymbol{\phi}(\mathbf{x}_n) \\ \tilde{\mathbf{w}}_1^T \boldsymbol{\phi}(\mathbf{x}_n) \\ \tilde{\mathbf{w}}_2^T \boldsymbol{\phi}(\mathbf{x}_n) \\ \tilde{\mathbf{w}}_3^T \boldsymbol{\phi}(\mathbf{x}_n) \\ \tilde{\mathbf{w}}_4^T \boldsymbol{\phi}(\mathbf{x}_n) \end{pmatrix}$$

$$\mathbf{W} = (\tilde{\mathbf{w}}_0, \tilde{\mathbf{w}}_1, \tilde{\mathbf{w}}_2, \tilde{\mathbf{w}}_3, \tilde{\mathbf{w}}_4)$$

- 对于N个数据，最小二乘损失函数为：

$$\|\mathbf{y}_n(\mathbf{x}) - \mathbf{t}_n\|^2 = \sum_{k=1}^{K} (y_{nk} - t_{nk})^2 = \sum_{k=1}^{K} \left(\tilde{\mathbf{w}}_k^T \boldsymbol{\phi}(\mathbf{x}_n) - t_{nk}\right)^2$$

$$Loss(\mathbf{W}) = \frac{1}{2} \sum_{n=1}^{N} \|\mathbf{y}_n(\mathbf{x}) - \mathbf{t}_n\|^2 = \frac{1}{2} \sum_{n=1}^{N} \sum_{k=1}^{K} \left(\tilde{\mathbf{w}}_k^T \boldsymbol{\phi}(\mathbf{x}_n) - t_{nk}\right)^2 = \frac{1}{2} \sum_{k=1}^{K} \sum_{n=1}^{N} \left(\tilde{\mathbf{w}}_k^T \boldsymbol{\phi}(\mathbf{x}_n) - t_{nk}\right)^2$$

- 观察 $\boldsymbol{\Phi}\mathbf{W} - \mathbf{T}$

$$\boldsymbol{\Phi}\mathbf{W} = \underbrace{\begin{pmatrix} \phi_0(x_1) & \phi_1(x_1) & \cdots & \phi_M(x_1) \\ \phi_0(x_2) & \phi_1(x_2) & \cdots & \phi_M(x_2) \\ \vdots & \vdots & \ddots & \vdots \\ \phi_0(x_N) & \phi_1(x_N) & \cdots & \phi_M(x_N) \end{pmatrix}}_{\boldsymbol{\phi}(x_n)^T \ \text{N×M}} \underbrace{\begin{pmatrix} w_{10} & w_{20} & \cdots & w_{K0} \\ w_{11} & w_{21} & \cdots & w_{K1} \\ \vdots & \vdots & \ddots & \vdots \\ w_{1M} & w_{2M} & \cdots & w_{KM} \end{pmatrix}}_{\tilde{\mathbf{w}}_k \ \text{M×K}} = \underbrace{\begin{pmatrix} \tilde{\mathbf{w}}_1^T \boldsymbol{\phi}(x_1) & \tilde{\mathbf{w}}_2^T \boldsymbol{\phi}(x_1) & \cdots & \tilde{\mathbf{w}}_K^T \boldsymbol{\phi}(x_1) \\ \tilde{\mathbf{w}}_1^T \boldsymbol{\phi}(x_2) & \tilde{\mathbf{w}}_2^T \boldsymbol{\phi}(x_2) & \cdots & \tilde{\mathbf{w}}_K^T \boldsymbol{\phi}(x_2) \\ \vdots & \vdots & \ddots & \vdots \\ \tilde{\mathbf{w}}_1^T \boldsymbol{\phi}(x_N) & \tilde{\mathbf{w}}_2^T \boldsymbol{\phi}(x_N) & \cdots & \tilde{\mathbf{w}}_K^T \boldsymbol{\phi}(x_N) \end{pmatrix}}_{\text{N×K}} = \underbrace{\begin{pmatrix} \left(\mathbf{y}_1(\mathbf{x}_1)\right)^T \\ \left(\mathbf{y}_2(\mathbf{x}_2)\right)^T \\ \cdots \\ \left(\mathbf{y}_N(\mathbf{x}_N)\right)^T \end{pmatrix}}_{\text{N×K}}$$

最小二乘求解判别函数

$$\boldsymbol{\Phi}\mathbf{W} = \underbrace{\begin{pmatrix} \phi_0(x_1) & \phi_1(x_1) & \cdots & \phi_M(x_1) \\ \phi_0(x_2) & \phi_1(x_2) & \cdots & \phi_M(x_2) \\ \vdots & \vdots & \ddots & \vdots \\ \phi_0(x_N) & \phi_1(x_N) & \cdots & \phi_M(x_N) \end{pmatrix}}_{\boldsymbol{\phi}(x_n)^T} \underbrace{\begin{pmatrix} w_{10} & w_{20} & \cdots & w_{K0} \\ w_{11} & w_{21} & \cdots & w_{K1} \\ \vdots & \vdots & \ddots & \vdots \\ w_{1M} & w_{2M} & \cdots & w_{KM} \end{pmatrix}}_{\widetilde{\mathbf{w}}_k} = \begin{pmatrix} \widetilde{\mathbf{w}}_1^T\boldsymbol{\phi}(x_1) & \widetilde{\mathbf{w}}_2^T\boldsymbol{\phi}(x_1) & \cdots & \widetilde{\mathbf{w}}_K^T\boldsymbol{\phi}(x_1) \\ \widetilde{\mathbf{w}}_1^T\boldsymbol{\phi}(x_2) & \widetilde{\mathbf{w}}_2^T\boldsymbol{\phi}(x_2) & \cdots & \widetilde{\mathbf{w}}_K^T\boldsymbol{\phi}(x_2) \\ \vdots & \vdots & \ddots & \vdots \\ \widetilde{\mathbf{w}}_1^T\boldsymbol{\phi}(x_N) & \widetilde{\mathbf{w}}_2^T\boldsymbol{\phi}(x_N) & \cdots & \widetilde{\mathbf{w}}_K^T\boldsymbol{\phi}(x_N) \end{pmatrix}$$

N×M　　M×K　　N×K

$$\boldsymbol{\Phi}\mathbf{W} - \mathbf{T} = \underbrace{\begin{pmatrix} \widetilde{\mathbf{w}}_1^T\boldsymbol{\phi}(x_1) & \widetilde{\mathbf{w}}_2^T\boldsymbol{\phi}(x_1) & \cdots & \widetilde{\mathbf{w}}_K^T\boldsymbol{\phi}(x_1) \\ \widetilde{\mathbf{w}}_1^T\boldsymbol{\phi}(x_2) & \widetilde{\mathbf{w}}_2^T\boldsymbol{\phi}(x_2) & \cdots & \widetilde{\mathbf{w}}_K^T\boldsymbol{\phi}(x_2) \\ \vdots & \vdots & \ddots & \vdots \\ \widetilde{\mathbf{w}}_1^T\boldsymbol{\phi}(x_N) & \widetilde{\mathbf{w}}_2^T\boldsymbol{\phi}(x_N) & \cdots & \widetilde{\mathbf{w}}_K^T\boldsymbol{\phi}(x_N) \end{pmatrix}}_{\text{N×K}} - \underbrace{\begin{pmatrix} \mathbf{t}_1^T \\ \mathbf{t}_2^T \\ \vdots \\ \mathbf{t}_N^T \end{pmatrix}}_{\text{N×K}} = \begin{pmatrix} \widetilde{\mathbf{w}}_1^T\boldsymbol{\phi}(x_1) - t_{11} & \widetilde{\mathbf{w}}_2^T\boldsymbol{\phi}(x_1) - t_{12} & \cdots & \widetilde{\mathbf{w}}_K^T\boldsymbol{\phi}(x_1) - t_{1K} \\ \widetilde{\mathbf{w}}_1^T\boldsymbol{\phi}(x_2) - t_{21} & \widetilde{\mathbf{w}}_2^T\boldsymbol{\phi}(x_2) - t_{22} & \cdots & \widetilde{\mathbf{w}}_K^T\boldsymbol{\phi}(x_2) - t_{2K} \\ \vdots & \vdots & \ddots & \vdots \\ \widetilde{\mathbf{w}}_1^T\boldsymbol{\phi}(x_N) - t_{N1} & \widetilde{\mathbf{w}}_2^T\boldsymbol{\phi}(x_N) - t_{N2} & \cdots & \widetilde{\mathbf{w}}_K^T\boldsymbol{\phi}(x_N) - t_{NK} \end{pmatrix}$$

$$\mathbf{t}_n = \begin{pmatrix} t_{n1} \\ t_{n2} \\ \vdots \\ t_{nK} \end{pmatrix} \qquad \begin{pmatrix} \mathbf{t}_1^T \\ \mathbf{t}_2^T \\ \vdots \\ \mathbf{t}_N^T \end{pmatrix} = \begin{pmatrix} t_{11} & t_{12} & \cdots & t_{1K} \\ t_{21} & t_{22} & \cdots & t_{2K} \\ \vdots & \vdots & \ddots & \vdots \\ t_{N1} & t_{N2} & \cdots & t_{NK} \end{pmatrix}$$

最小二乘求解判别函数

$$\mathbf{\Phi W}-\mathbf{T}=\begin{pmatrix}\tilde{\mathbf{w}}_1^T\boldsymbol{\phi}(x_1) & \tilde{\mathbf{w}}_2^T\boldsymbol{\phi}(x_1) & \cdots & \tilde{\mathbf{w}}_K^T\boldsymbol{\phi}(x_1)\\ \tilde{\mathbf{w}}_1^T\boldsymbol{\phi}(x_2) & \tilde{\mathbf{w}}_2^T\boldsymbol{\phi}(x_2) & \cdots & \tilde{\mathbf{w}}_K^T\boldsymbol{\phi}(x_2)\\ \vdots & \vdots & \ddots & \vdots\\ \tilde{\mathbf{w}}_1^T\boldsymbol{\phi}(x_N) & \tilde{\mathbf{w}}_2^T\boldsymbol{\phi}(x_N) & \cdots & \tilde{\mathbf{w}}_K^T\boldsymbol{\phi}(x_N)\end{pmatrix}_{N\times K}-\begin{pmatrix}\mathbf{t}_1^T\\ \mathbf{t}_2^T\\ \vdots\\ \mathbf{t}_N^T\end{pmatrix}_{N\times K}=\begin{pmatrix}\tilde{\mathbf{w}}_1^T\boldsymbol{\phi}(x_1)-t_{11} & \tilde{\mathbf{w}}_2^T\boldsymbol{\phi}(x_1)-t_{12} & \cdots & \tilde{\mathbf{w}}_K^T\boldsymbol{\phi}(x_1)-t_{1K}\\ \tilde{\mathbf{w}}_1^T\boldsymbol{\phi}(x_2)-t_{21} & \tilde{\mathbf{w}}_2^T\boldsymbol{\phi}(x_2)-t_{22} & \cdots & \tilde{\mathbf{w}}_K^T\boldsymbol{\phi}(x_2)-t_{2K}\\ \vdots & \vdots & \ddots & \vdots\\ \tilde{\mathbf{w}}_1^T\boldsymbol{\phi}(x_N)-t_{N1} & \tilde{\mathbf{w}}_2^T\boldsymbol{\phi}(x_N)-t_{N2} & \cdots & \tilde{\mathbf{w}}_K^T\boldsymbol{\phi}(x_N)-t_{NK}\end{pmatrix}$$

- 最小二乘损失函数：

$$Loss(\mathbf{W})=\frac{1}{2}\sum_{n=1}^{N}\|\hat{\mathbf{y}}_n(\mathbf{x})-\mathbf{t}_n\|^2=\frac{1}{2}\sum_{n=1}^{N}\sum_{k=1}^{K}\left(\tilde{\mathbf{w}}_k^T\boldsymbol{\phi}(\mathbf{x}_n)-t_{nk}\right)^2=\frac{1}{2}\sum_{k=1}^{K}\sum_{n=1}^{N}\left(\tilde{\mathbf{w}}_k^T\boldsymbol{\phi}(\mathbf{x}_n)-t_{nk}\right)^2$$

- **观察$(\mathbf{\Phi W}-\mathbf{T})^T(\mathbf{\Phi W}-\mathbf{T})$的主对角线元素（矩阵大小为K×K）**

$$(\mathbf{\Phi W}-\mathbf{T})^T(\mathbf{\Phi W}-\mathbf{T})_{11}=\sum_{n=1}^{N}(\tilde{\mathbf{w}}_1^T\boldsymbol{\phi}(\mathbf{x}_n)-t_{n1})^2,\qquad(\mathbf{\Phi W}-\mathbf{T})^T(\mathbf{\Phi W}-\mathbf{T})_{22}=\sum_{n=1}^{N}(\tilde{\mathbf{w}}_2^T\boldsymbol{\phi}(\mathbf{x}_n)-t_{n2})^2$$

$$(\mathbf{\Phi W}-\mathbf{T})^T(\mathbf{\Phi W}-\mathbf{T})_{kk}=\sum_{n=1}^{N}\left(\tilde{\mathbf{w}}_k^T\boldsymbol{\phi}(\mathbf{x}_n)-t_{nk}\right)^2$$

最小二乘求解判别函数

- 最小二乘损失函数：

$$Loss(\mathbf{W}) = \frac{1}{2}\sum_{n=1}^{N}\|\hat{\mathbf{y}}_n(\mathbf{x}) - \mathbf{t}_n\|^2 = \frac{1}{2}\sum_{n=1}^{N}\sum_{k=1}^{K}\left(\tilde{\mathbf{w}}_k^T\boldsymbol{\phi}(\mathbf{x}_n) - t_{nk}\right)^2 = \frac{1}{2}\sum_{k=1}^{K}\sum_{n=1}^{N}\left(\tilde{\mathbf{w}}_k^T\boldsymbol{\phi}(\mathbf{x}_n) - t_{nk}\right)^2$$

- **为$(\mathbf{\Phi W} - \mathbf{T})^T(\mathbf{\Phi W} - \mathbf{T})$的主对角线元素之和**

$$(\mathbf{\Phi W} - \mathbf{T})^T(\mathbf{\Phi W} - \mathbf{T})_{kk} = \sum_{n=1}^{N}\left(\tilde{\mathbf{w}}_k^T\boldsymbol{\phi}(\mathbf{x}_n) - t_{nk}\right)^2$$

$$Loss(\mathbf{W}) = \frac{1}{2}\mathrm{Tr}\{(\mathbf{\Phi W} - \mathbf{T})^T(\mathbf{\Phi W} - \mathbf{T})\}$$

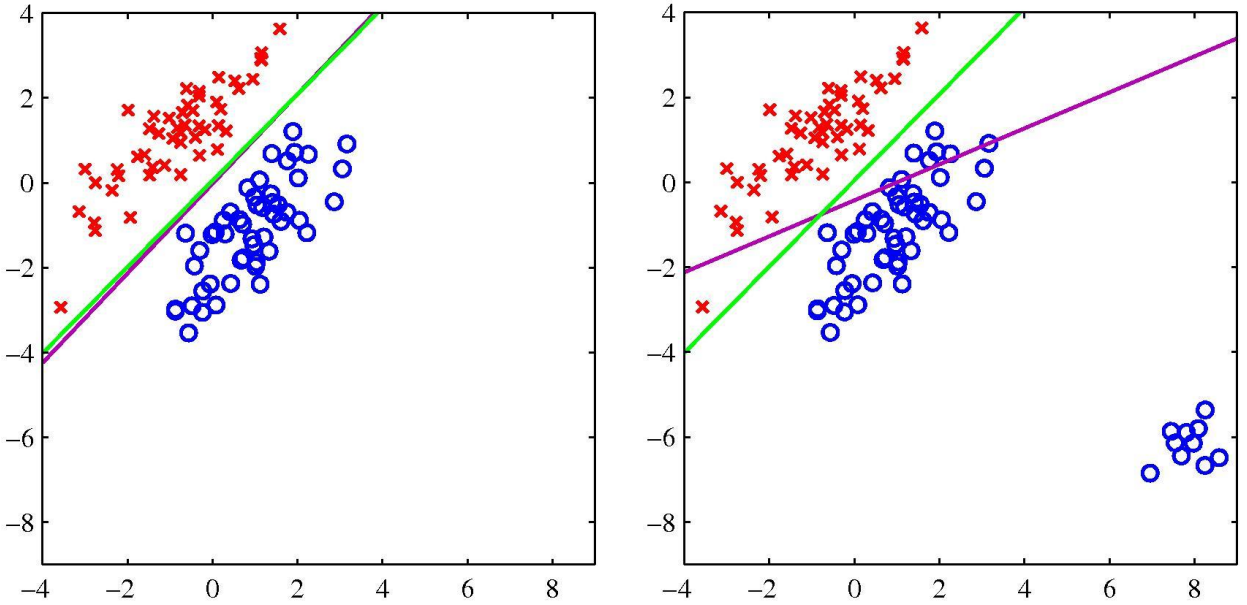
- **对损失函数求导，令其=0，可得$\mathbf{W} = (\mathbf{\Phi}^T\mathbf{\Phi})^{-1}\mathbf{\Phi}^T\mathbf{T}$**
- **形式上与线性回归$\mathbf{w} = (\mathbf{\Phi}^T\mathbf{\Phi})^{-1}\mathbf{\Phi}^T\mathbf{t}$相同，只是t变为T， w变为W**

最小二乘求解判别函数

- 最小二乘判别函数的参数

$$\mathbf{W} = (\mathbf{\Phi}^T\mathbf{\Phi})^{-1}\mathbf{\Phi}^T\mathbf{T}$$

- 对于离群数据极为敏感



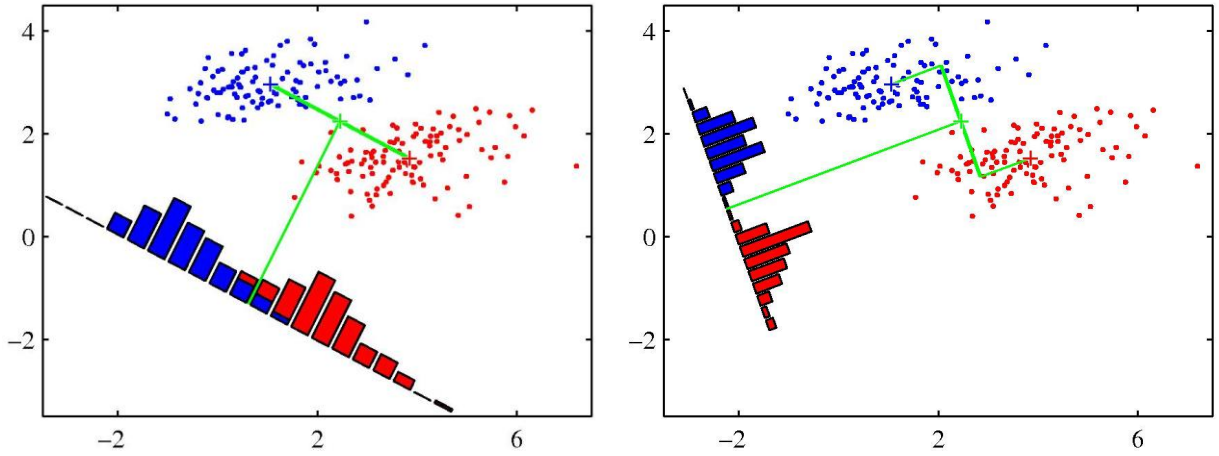
Fisher判别函数（LDA）

“投影”回想向量内积的几何意义

- 把分类问题理解为降维问题，把x降维到$y(\mathbf{x}) = \mathbf{w}^T\mathbf{x}$，$\mathbf{x} \in \begin{cases} C1, & y(\mathbf{x}) > -w_0 \\ C2, & otherwise \end{cases}$
- 几何意义：
  - 高维降维到1维后，可能会存在严重的混叠
  - LDA就是找到能够是混叠最小的那个降维方式，即分类间隔最大。

左图：

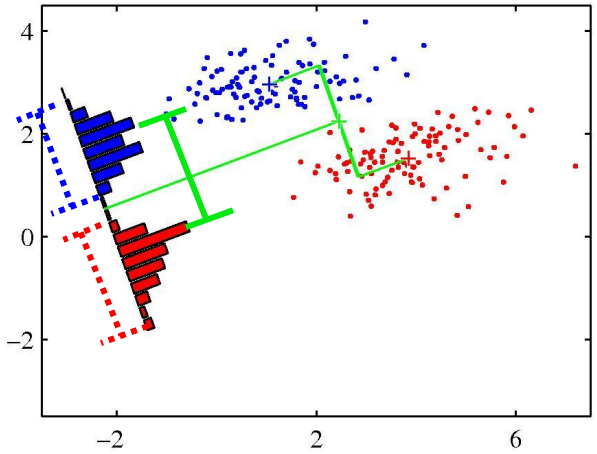
x为2维，投影方向w为两类均值的连线方向，混叠非常严重。



右图：

x为2维，投影方向w由LDA确定，混叠非常小。

Fisher判别函数（LDA）



- 假定数据集C1类有N1个样本，C2类有N2个样本，均值向量为：

$$\mathbf{m}_1 = \frac{\mathbf{1}}{N_1}\sum_{n\varepsilon C_1}\mathbf{x}_n\,,\mathbf{m}_2 = \frac{\mathbf{1}}{N_2}\sum_{n\varepsilon C_2}\mathbf{x}_n$$

- LDA选择的w能够使投影后的两类数据的均值的距离最大化，即最大化下式

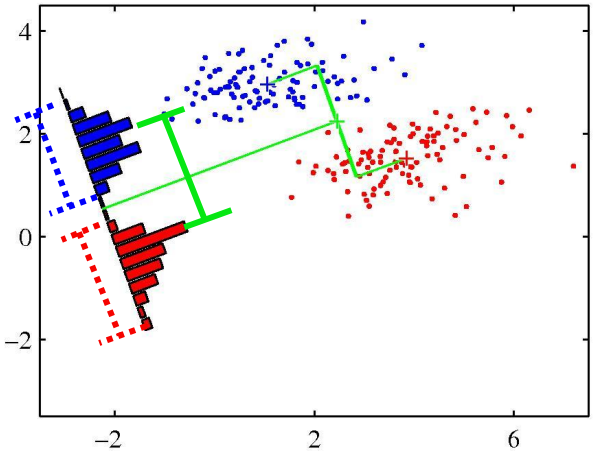
$$m_1 - m_2 = \mathbf{w}^T\mathbf{m}_1 - \mathbf{w}^T\mathbf{m}_2 = \mathbf{w}^T(\mathbf{m}_1 - \mathbf{m}_2)$$

- 如果w不做限制，上式可以无限大，而我们关系的只是其方向，因此可限制$\sum_i w_i^2 = 1$
- 此外，还要限制投影后的类内方差，使其最小，这样才能最好的避免类间混叠。

Fisher判别函数（LDA）

- 投影后，每个类的类内方差计算如下：

$$s_k{}^2 = \sum_{n\varepsilon C_k} (\mathbf{w}^T\mathbf{x}_n - m_k)^2$$



- 总内类方差为： $s_1{}^2 + s_2{}^2$
- LDA要优化的目标函数为：

$$J(\mathbf{w}) = \frac{(m_1 - m_2)^2}{s_1{}^2 + s_2{}^2} = \frac{\mathbf{w}^T(\mathbf{m}_1 - \mathbf{m}_2)\cdot(\mathbf{m}_1 - \mathbf{m}_2)^T\mathbf{w}}{\sum_{n\varepsilon C_1}(\mathbf{w}^T\mathbf{x}_n - \mathbf{w}^T\mathbf{m}_1)^2 + \sum_{n\varepsilon C_2}(\mathbf{w}^T\mathbf{x}_n - \mathbf{w}^T\mathbf{m}_2)^2}$$

$$= \frac{\mathbf{w}^T(\mathbf{m}_1 - \mathbf{m}_2)\cdot(\mathbf{m}_1 - \mathbf{m}_2)^T\mathbf{w}}{\sum_{n\varepsilon C_1}\mathbf{w}^T(\mathbf{x}_n - \mathbf{m}_1)\cdot(\mathbf{x}_n - \mathbf{m}_1)^T\mathbf{w} + \sum_{n\varepsilon C_2}\mathbf{w}^T(\mathbf{x}_n - \mathbf{m}_2)\cdot(\mathbf{x}_n - \mathbf{m}_2)^T\mathbf{w}}$$

$$= \frac{\mathbf{w}^T\left((\mathbf{m}_1 - \mathbf{m}_2)\cdot(\mathbf{m}_1 - \mathbf{m}_2)^T\right)\mathbf{w}}{\mathbf{w}^T\left(\sum_{n\varepsilon C_1}(\mathbf{x}_n - \mathbf{m}_1)\cdot(\mathbf{x}_n - \mathbf{m}_1)^T + \sum_{n\varepsilon C_2}(\mathbf{x}_n - \mathbf{m}_2)\cdot(\mathbf{x}_n - \mathbf{m}_2)^T\right)\mathbf{w}} = \frac{\mathbf{w}^T\mathrm{S_B}\mathbf{w}}{\mathbf{w}^T\mathrm{S_W}\mathbf{w}}$$

类间协方差阵

类内协方差阵

Fisher判别函数（LDA）

- LDA要优化的目标函数为：

$$J(\mathbf{w}) = \frac{\mathbf{w}^T\left((\mathbf{m}_1 - \mathbf{m}_2)\cdot(\mathbf{m}_1 - \mathbf{m}_2)^T\right)\mathbf{w}}{\mathbf{w}^T\left(\sum_{n\varepsilon C_1}(\mathbf{x}_n - \mathbf{m}_1)\cdot(\mathbf{x}_n - \mathbf{m}_1)^T + \sum_{n\varepsilon C_2}(\mathbf{x}_n - \mathbf{m}_2)\cdot(\mathbf{x}_n - \mathbf{m}_2)^T\right)\mathbf{w}} = \frac{\mathbf{w}^T \mathrm{S_B}\mathbf{w}}{\mathbf{w}^T \mathrm{S_W}\mathbf{w}}$$

类间协方差阵

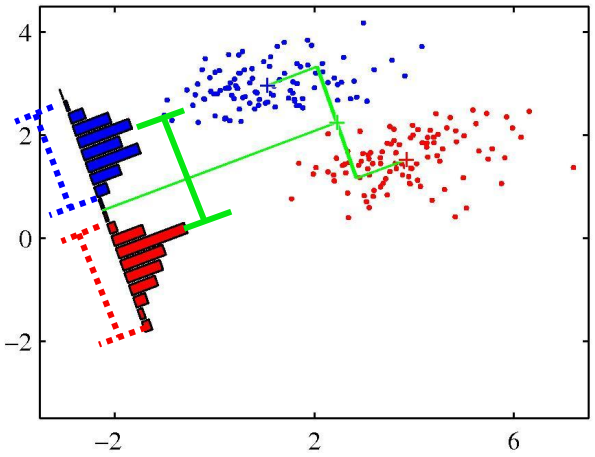
类内协方差阵

- 上式对$\mathbf{w}$求导，可得$J(\mathbf{w})$最大的条件为：$(\mathbf{w}^T \mathrm{S_B}\mathbf{w})\mathrm{S_W}\mathbf{w} = (\mathbf{w}^T \mathrm{S_W}\mathbf{w})\mathrm{S_B}\mathbf{w}$

$$\mathrm{S_B}\mathbf{w} = (\mathbf{m}_1 - \mathbf{m}_2)\cdot\underbrace{(\mathbf{m}_1 - \mathbf{m}_2)^T\mathbf{w}}_{\text{标量}}$$

- **可知**$\mathrm{S_B}\mathbf{w}$与$(\mathbf{m}_1 - \mathbf{m}_2)$方向一致
- 由于我们只关心$\mathbf{w}$的方向，我们可以忽略另外两个标量因子$\mathbf{w}^T \mathrm{S_B}\mathbf{w}$和$\mathbf{w}^T \mathrm{S_W}\mathbf{w}$
- $J(\mathbf{w})$最大化条件两侧都乘以$\mathrm{S_W^{-1}}$，可得$\mathbf{w} \propto \mathrm{S_W^{-1}}\ \mathrm{S_B}\mathbf{w} \propto \mathrm{S_W^{-1}}\ (\mathbf{m}_1 - \mathbf{m}_2)$

$$\mathbf{w} \propto \mathrm{S_W^{-1}}\ (\mathbf{m}_1 - \mathbf{m}_2)$$



Fisher判别函数（LDA）

$$\mathbf{w} \propto S_W^{-1}(\mathbf{m}_1 - \mathbf{m}_2)$$

$$t = \{0,1\} \longrightarrow t = \left\{-\frac{N}{N_2}, \frac{N}{N_1}\right\}$$

- LDA与最小二乘的关系?
- 对于两类问题，用正值$\frac{N}{N_1}$表示C1的t值，负值$-\frac{N}{N_2}$表示C2的t值。两类问题只需一组$\mathbf{w}$。

$$Loss(\mathbf{w}) = \frac{1}{2}\sum_{n=1}^{N}\|y_n(\mathbf{x}_n) - t_n\|^2 = \frac{1}{2}\sum_{n=1}^{N}(\mathbf{w}^T\mathbf{x}_n + w_0 - t_n)^2$$

- 求解$\mathbf{w}$

$$\frac{dLoss(w_0)}{dw_0} = \sum_{n=1}^{N}(\mathbf{w}^T\mathbf{x}_n + w_0 - t_n) = 0 \qquad \frac{dLoss(\mathbf{w})}{d\mathbf{w}} = \sum_{n=1}^{N}(\mathbf{w}^T\mathbf{x}_n + w_0 - t_n)\mathbf{x}_n = 0$$

Fisher判别函数（LDA）

$$\mathbf{w} \propto S_W^{-1}(\mathbf{m}_1 - \mathbf{m}_2)$$

- LDA求解

$$Loss(\mathbf{w}) = \frac{1}{2}\sum_{n=1}^{N}\|y_n(\mathbf{x}_n) - t_n\|^2 = \frac{1}{2}\sum_{n=1}^{N}(\mathbf{w}^T\mathbf{x}_n + w_0 - t_n)^2$$

$$\frac{dLoss(w_0)}{dw_0} = \sum_{n=1}^{N}(\mathbf{w}^T\mathbf{x}_n + w_0 - t_n) = 0 \qquad \frac{dLoss(\mathbf{w})}{d\mathbf{w}} = \sum_{n=1}^{N}(\mathbf{w}^T\mathbf{x}_n + w_0 - t_n)\mathbf{x}_n = 0$$

$$\sum_{n=1}^{N}(\mathbf{w}^T\mathbf{x}_n + w_0 - t_n) = 0 \qquad \sum_{n=1}^{N} t_n = N_1\frac{N}{N_1} - N_2\frac{N}{N_2} = 0$$

$$w_0 = -\frac{1}{N}\sum_{n=1}^{N}\mathbf{w}^T\mathbf{x}_n = -\mathbf{w}^T\sum_{n=1}^{N}\frac{\mathbf{x}_n}{N} = -\mathbf{w}^T\mathbf{m}$$

$$\frac{dLoss(\mathbf{w})}{d\mathbf{w}} = \sum_{n=1}^{N}(\mathbf{w}^T\mathbf{x}_n - \mathbf{w}^T\mathbf{m} - t_n)\mathbf{x}_n = 0$$

Fisher判别函数（LDA）

$$\frac{dLoss(\mathbf{w})}{d\mathbf{w}} = \sum_{n=1}^{N}(\mathbf{w}^T\mathbf{x}_n - \mathbf{w}^T\mathbf{m} - t_n)\mathbf{x}_n = \sum_{n=1}^{N}(\mathbf{x}_n\mathbf{x}_n{}^T\mathbf{w} - \mathbf{x}_n\mathbf{m}^T\mathbf{w} - \mathbf{x}_n t_n) = \sum_{n=1}^{N}\left((\mathbf{x}_n\mathbf{x}_n{}^T - \mathbf{x}_n\mathbf{m}^T)\mathbf{w} - \mathbf{x}_n t_n\right)$$

$$= \sum_{n\in C_1}^{N_1}\left((\mathbf{x}_n\mathbf{x}_n{}^T - \mathbf{x}_n\mathbf{m}^T)\mathbf{w} - \mathbf{x}_n\frac{N}{N_1}\right) + \sum_{m\in C_2}^{N_2}\left((\mathbf{x}_m\mathbf{x}_m{}^T - \mathbf{x}_m\mathbf{m}^T)\mathbf{w} + \mathbf{x}_m\frac{N}{N_2}\right)$$

$$= \sum_{n\in C_1}^{N_1}(\mathbf{x}_n\mathbf{x}_n{}^T)\mathbf{w} - \sum_{n\in C_1}^{N_1}(\mathbf{x}_n\mathbf{m}^T)\mathbf{w} - \sum_{n\in C_1}^{N_1}\mathbf{x}_n\frac{N}{N_1} + \sum_{m\in C_2}^{N_2}(\mathbf{x}_m\mathbf{x}_m{}^T)\mathbf{w} - \sum_{m\in C_2}^{N_2}(\mathbf{x}_m\mathbf{m}^T)\mathbf{w} + \sum_{m\in C_2}^{N_2}\left(\mathbf{x}_m\frac{N}{N_2}\right)$$

$$= \left(\sum_{n\in C_1}^{N_1}(\mathbf{x}_n\mathbf{x}_n{}^T) - \sum_{n\in C_1}^{N_1}(\mathbf{x}_n\mathbf{m}^T)\right)\mathbf{w} - N_1\mathbf{m}_1\frac{N}{N_1} + \left(\sum_{m\in C_2}^{N_2}(\mathbf{x}_m\mathbf{x}_m{}^T) - \sum_{m\in C_2}^{N_2}(\mathbf{x}_m\mathbf{m}^T)\right)\mathbf{w} + N_2\mathbf{m}_2\frac{N}{N_2}$$

$$= \left(\sum_{n\in C_1}^{N_1}(\mathbf{x}_n\mathbf{x}_n{}^T) - N_1\mathbf{m}_1\mathbf{m}^T\right)\mathbf{w} - N_1\mathbf{m}_1\frac{N}{N_1} + \left(\sum_{m\in C_2}^{N_2}(\mathbf{x}_m\mathbf{x}_m{}^T) - N_2\mathbf{m}_2\mathbf{m}^T\right)\mathbf{w} + N_2\mathbf{m}_2\frac{N}{N_2}$$

$$= \left(\sum_{n\in C_1}^{N_1}(\mathbf{x}_n\mathbf{x}_n{}^T) + \sum_{m\in C_2}^{N_2}(\mathbf{x}_m\mathbf{x}_m{}^T) - N_1\mathbf{m}_1\mathbf{m}^T - N_2\mathbf{m}_2\mathbf{m}^T\right)\mathbf{w} - N_1\mathbf{m}_1\frac{N}{N_1} + N_2\mathbf{m}_2\frac{N}{N_2}$$

Fisher判别函数（LDA）

$$S_W = \sum_{n\varepsilon C_1} (\mathbf{x}_n - \mathbf{m}_1) \cdot (\mathbf{x}_n - \mathbf{m}_1)^T + \sum_{n\varepsilon C_2} (\mathbf{x}_n - \mathbf{m}_2) \cdot (\mathbf{x}_n - \mathbf{m}_2)^T$$

$$\sum_{n\varepsilon C_k} (\mathbf{x}_n - \mathbf{m}_k) \cdot (\mathbf{x}_n - \mathbf{m}_k)^T = \sum_{n\in C_k}^{N_k} (\mathbf{x}_n \mathbf{x}_n{}^T - \mathbf{x}_n \mathbf{m}_k{}^T - \mathbf{m}_k \mathbf{x}_n{}^T + \mathbf{m}_k \mathbf{m}_k{}^T)$$

$$= \sum_{n\in C_k}^{N_k} \mathbf{x}_n \mathbf{x}_n{}^T - \sum_{n\in C_k}^{N_k} \mathbf{x}_n \mathbf{m}_k{}^T - \sum_{n\in C_k}^{N_k} \mathbf{m}_k \mathbf{x}_n{}^T + \sum_{n\in C_k}^{N_k} \mathbf{m}_k \mathbf{m}_k{}^T$$

$$= \sum_{n\in C_k}^{N_k} \mathbf{x}_n \mathbf{x}_n{}^T - N_k \mathbf{m}_k \mathbf{m}_k{}^T - N_k \mathbf{m}_k \mathbf{m}_k{}^T + N_k \mathbf{m}_k \mathbf{m}_k{}^T$$

$$= \sum_{n\in C_k}^{N_k} \mathbf{x}_n \mathbf{x}_n{}^T - N_k \mathbf{m}_k \mathbf{m}_k{}^T$$

Fisher判别函数（LDA）

$$S_W = \sum_{n\in C_1} \mathbf{x}_n \mathbf{x}_n{}^T - N_1 \mathbf{m}_1 \mathbf{m}_1{}^T + \sum_{m\in C_2} \mathbf{x}_m \mathbf{x}_m{}^T - N_2 \mathbf{m}_2 \mathbf{m}_2{}^T$$

$$S_B = (\mathbf{m}_1 - \mathbf{m}_2) \cdot (\mathbf{m}_1 - \mathbf{m}_2)^T$$

$$\frac{dLoss(\mathbf{w})}{d\mathbf{w}} = \sum_{n=1}^{N} (\mathbf{w}^T \mathbf{x}_n - \mathbf{w}^T \mathbf{m} - y_n)\mathbf{x}_n$$

$$\mathbf{m} = \frac{1}{N}\sum_{n=1}^{N} \mathbf{x}_n = \frac{1}{N}(N_1 \mathbf{m}_1 + N_2 \mathbf{m}_2)$$

$$= \left(\sum_{n\in C_1} (\mathbf{x}_n \mathbf{x}_n{}^T) + \sum_{m\in C_2} (\mathbf{x}_m \mathbf{x}_m{}^T) - (N_1 \mathbf{m}_1 + N_2 \mathbf{m}_2)\mathbf{m}^T \right) \mathbf{w} - N_1 \mathbf{m}_1 \frac{N}{N_1} + N_2 \mathbf{m}_2 \frac{N}{N_2}$$

$$= \left(S_W + N_1 \mathbf{m}_1 \mathbf{m}_1{}^T + N_2 \mathbf{m}_2 \mathbf{m}_2{}^T - (N_1 \mathbf{m}_1 + N_2 \mathbf{m}_2) \frac{1}{N} (N_1 \mathbf{m}_1 + N_2 \mathbf{m}_2)^T \right) \mathbf{w} - N(\mathbf{m}_1 - \mathbf{m}_2)$$

$$= \left(S_W + \frac{N_1 N_2}{N} (\mathbf{m}_1 \mathbf{m}_1{}^T - \mathbf{m}_1 \mathbf{m}_2{}^T - \mathbf{m}_2 \mathbf{m}_1{}^T - \mathbf{m}_2 \mathbf{m}_2{}^T) \right) \mathbf{w} - N(\mathbf{m}_1 - \mathbf{m}_2)$$

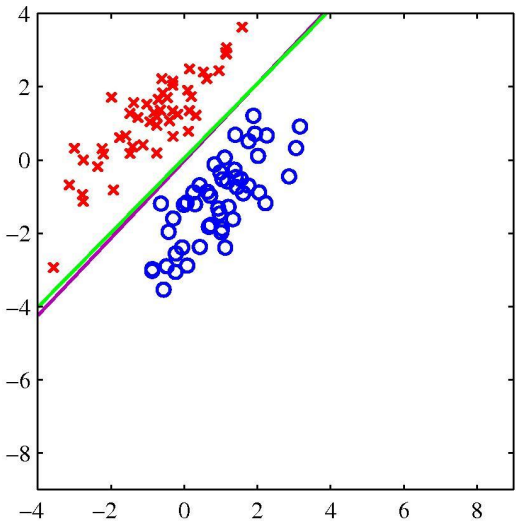
$$= \left(S_W + \frac{N_1 N_2}{N} S_B \right) \mathbf{w} - N(\mathbf{m}_1 - \mathbf{m}_2) = S_W \mathbf{w} - \alpha(\mathbf{m}_1 - \mathbf{m}_2) = 0 \rightarrow \mathbf{w} \propto S_W^{-1} (\mathbf{m}_1 - \mathbf{m}_2)$$

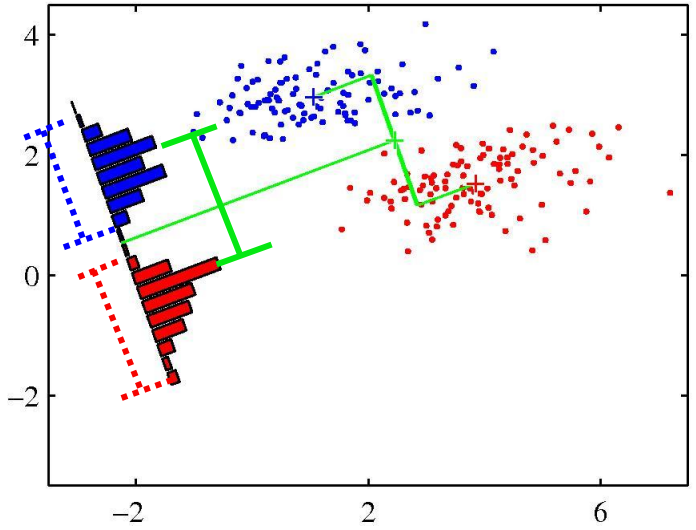
$$S_B \mathbf{w} = (\mathbf{m}_1 - \mathbf{m}_2) \cdot \underbrace{(\mathbf{m}_1 - \mathbf{m}_2)^T \mathbf{w}}_{\text{标量}} = \lambda(\mathbf{m}_1 - \mathbf{m}_2)$$

Fisher判别函数（LDA）

- 对于两类问题，用正值$\frac{N}{N_1}$表示C_1的t值，负值$-\frac{N}{N_2}$表示C_2的t值。
- Fisher判别函数(LDA) 和 最小二乘方法 等价

$$\mathbf{w}^* \propto \mathrm{S}_\mathrm{W}^{-1}\left(\mathbf{m}_1 - \mathbf{m}_2\right)$$





Fisher判别函数（LDA）

- 对于两类问题，用正值$\frac{N}{N_1}$表示C_1的t值，负值$-\frac{N}{N_2}$表示C_2的t值。
- Fisher 和最小二乘方法等价

$$\mathbf{w}^* \propto \mathrm{S}_\mathrm{W}^{-1}\left(\mathbf{m}_1 - \mathbf{m}_2\right)$$

- 多类问题，要优化的目标函数如下：

$$J(\mathbf{W}) = \mathrm{Tr}\{(\mathbf{W}\mathbf{S}_\mathrm{W}\mathbf{W}^T)^{-1}(\mathbf{W}\mathbf{S}_\mathrm{B}\mathbf{W}^T)\}$$

参数由$\mathbf{S}_\mathrm{W}{}^{-1}\mathbf{S}_\mathrm{B}$的特征向量决定

线性判别函数：最小二乘与LDA总结

1. 本质上类似

2. 对离群点非常敏感

3. 适合类内都是近似正态分布的数据

线性判别函数：引入激活机制——感知机

1. 到目前为止， t值的预测都是如下形式（含偏置项）

$$y(\mathbf{x}) = \mathbf{w}^T \boldsymbol{\phi}(\mathbf{x})$$

2. 为了更好的描述分类问题，对上述表达式进行修改，引入激活函数

$$y(\mathbf{x}) = f\left(\mathbf{w}^T \boldsymbol{\phi}(\mathbf{x})\right), where\ f(a) = \begin{cases} +1, a \geq 0 \\ -1, a < 0 \end{cases}$$

- 感知机算法中的数据目标值不用$t = \{0,1\}$ 表示 （0-1表达是为概率表达服务，感知机有概率版本）
- 而是用$t = \{-1,1\}$表示 （暂不考虑概率意义）

1. 感知机算法的损失函数由所有错分数据定义：

$$E_p(\mathbf{w}) = -\sum_n \boldsymbol{y}(\mathbf{x}_n)t_n = -\sum_n \mathbf{w}^T\boldsymbol{\phi}(\mathbf{x}_n)t_n$$

2. 损失函数意义：

预测错误的数据，y与t异号有损失；预测正确，y和t同号无损失。

- 预测值y与目标值t同号无损失
- 预测值y与目标值t异号有损失

$$y(\mathbf{x}) = f\left(\mathbf{w}^T\boldsymbol{\phi}(\mathbf{x})\right), where\ f(a) = \begin{cases} +1, a \geq 0 \\ -1, a < 0 \end{cases}$$

感知机

1. 感知机算法的损失函数由所有错分数据定义：

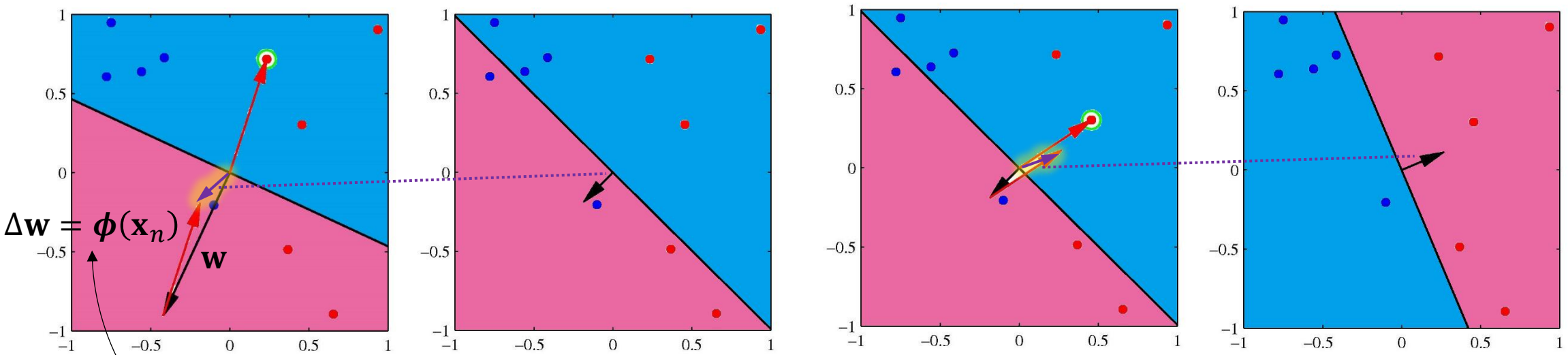
$$E_p(\mathbf{w}) = -\sum_n \boldsymbol{y}(\mathbf{x}_n)t_n = -\sum_n \mathbf{w}^T\boldsymbol{\phi}(\mathbf{x}_n)t_n$$

2. 如何求解？

- 所有错分数据带来的损失对w求导：$\frac{dE_p(\mathbf{w})}{d\mathbf{w}} = -\sum_n \boldsymbol{\phi}(\mathbf{x}_n)t_n$
- SGD（逐个加入错分数据计算损失，更新权值）：$\Delta\mathbf{w} = -\eta\nabla E_p(\mathbf{w}) = -\eta(-\boldsymbol{\phi}(\mathbf{x}_n)t_n) = \eta\boldsymbol{\phi}(\mathbf{x}_n)t_n$
- 如果数据是线性可分的，则SGD一定能够收敛到其中一个正确解
- 可以分开两类的解可能有多个，最终找到哪个跟w初始值有关系

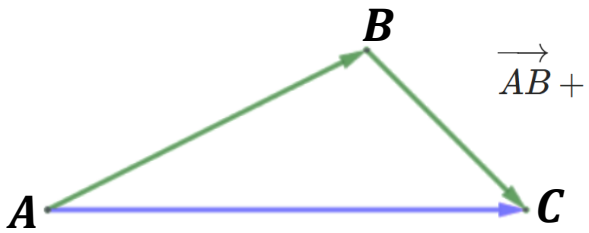
感知机

- SGD（逐个加入错分数据，计算损失，更新权值）：$\Delta \mathbf{w} = -\eta \nabla E_p(\mathbf{w}) = -\eta(-\boldsymbol{\phi}(\mathbf{x}_n)t_n) = \eta \boldsymbol{\phi}(\mathbf{x}_n)t_n$
- 红色数据点为C1（$t = 1$），蓝色数据点为C2（$t = -1$），$\eta = 1$
- 决策边界为黑色，黑色箭头指向判断为C1的区域，黑色箭头反方向为判断为C2的区域。



$$\Delta \mathbf{w} = \boldsymbol{\phi}(\mathbf{x}_n)t_n = \boldsymbol{\phi}(\mathbf{x}_n)1$$

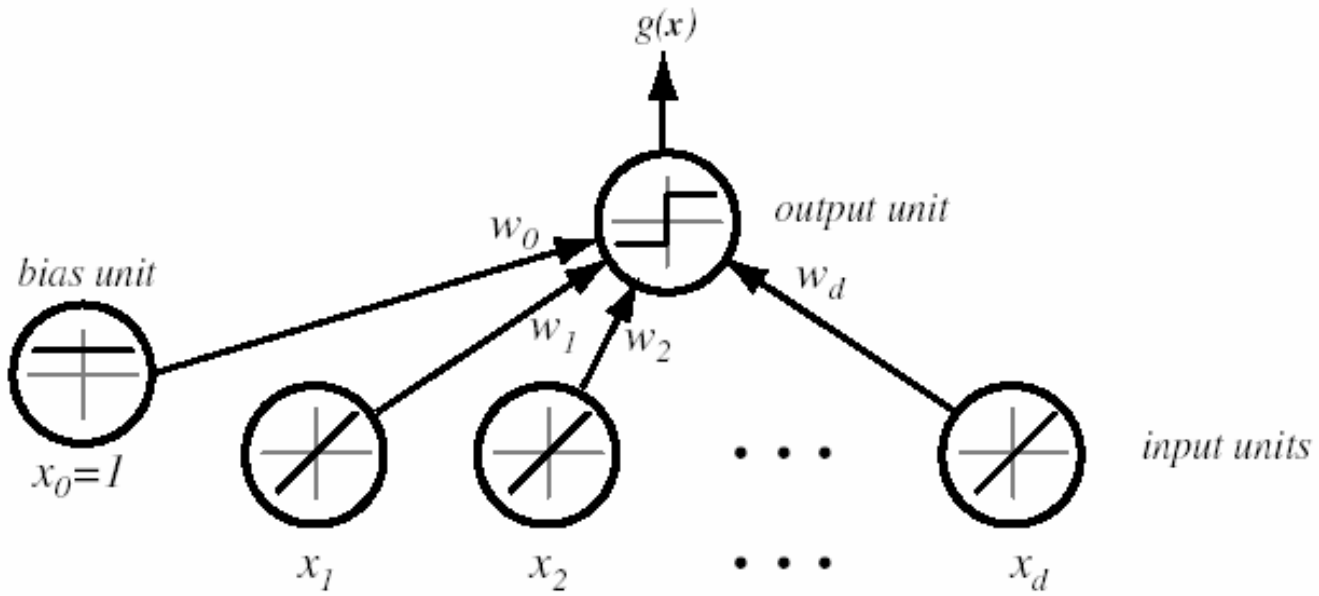
向量加法，三角形法则



$$\overrightarrow{AB} + \overrightarrow{BC} = \overrightarrow{AC}$$

感知机

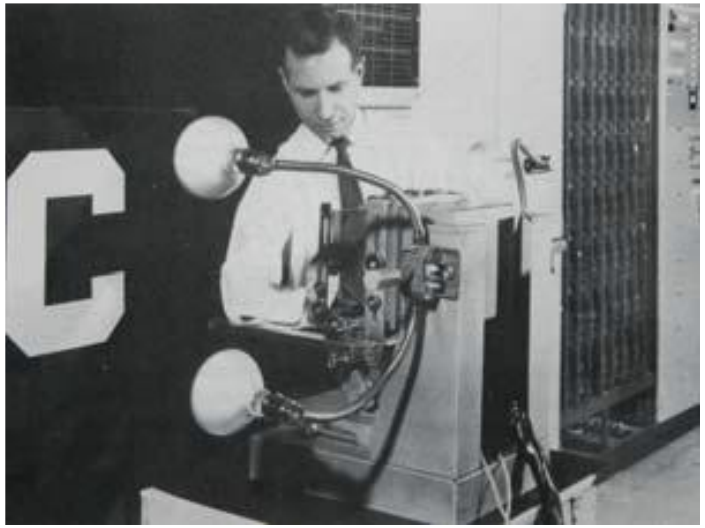
- 典型的神经元结构，在线性函数基础上进行非线性激活，线性函数有偏置项。



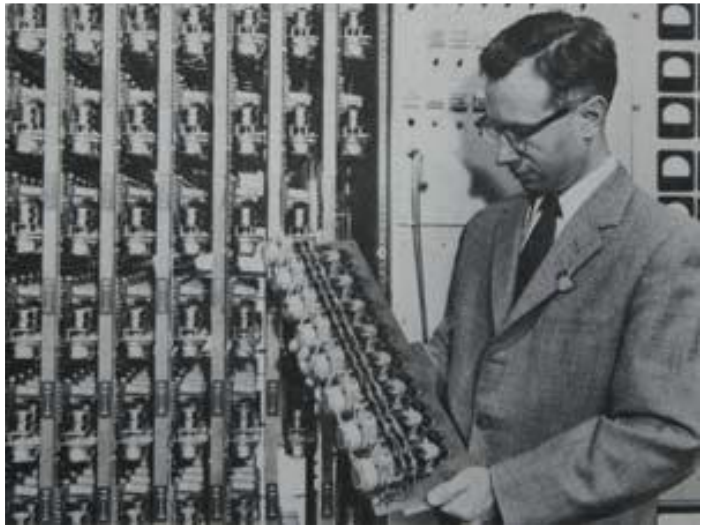
- 感知机选择的激活函数和目标值，没有概率解释。稍加修改就可以，构成神经网络的基础。

感知机

- 第一次人工智能高潮，并行计算模拟电路硬件，字符识别，然好景不长
- 因不能解决线性不可分问题（如异或），淹没在若干大佬的批评声中，神经网络背锅
- 上世纪80年代中期，发现无法解决线性不可分问题，只是因为使用了单个或单层神经元（感知机）

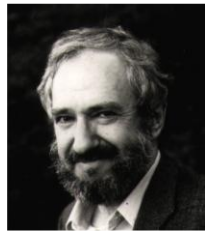






Rosenblatt, Frank, The Perceptron--a perceiving and recognizing automaton. Report 85-460-1, 1957， Cornell Aeronautical Laboratory



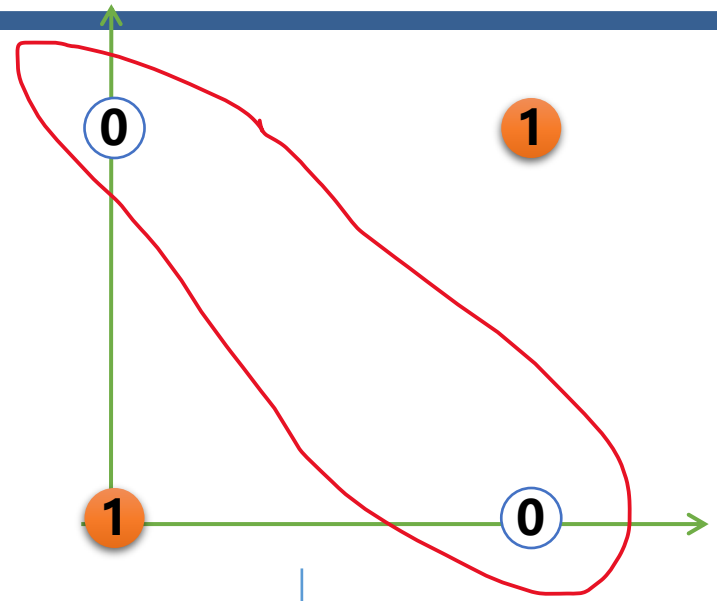


Minsky and Papert

Minsky M. L. and Papert S. A. 1969, *Perceptrons*, MIT Press

感知机

不能解决线性不可分问题（如异或）



OR

x_1	x_2	t
0	0	0
0	1	1
1	0	1
1	1	1

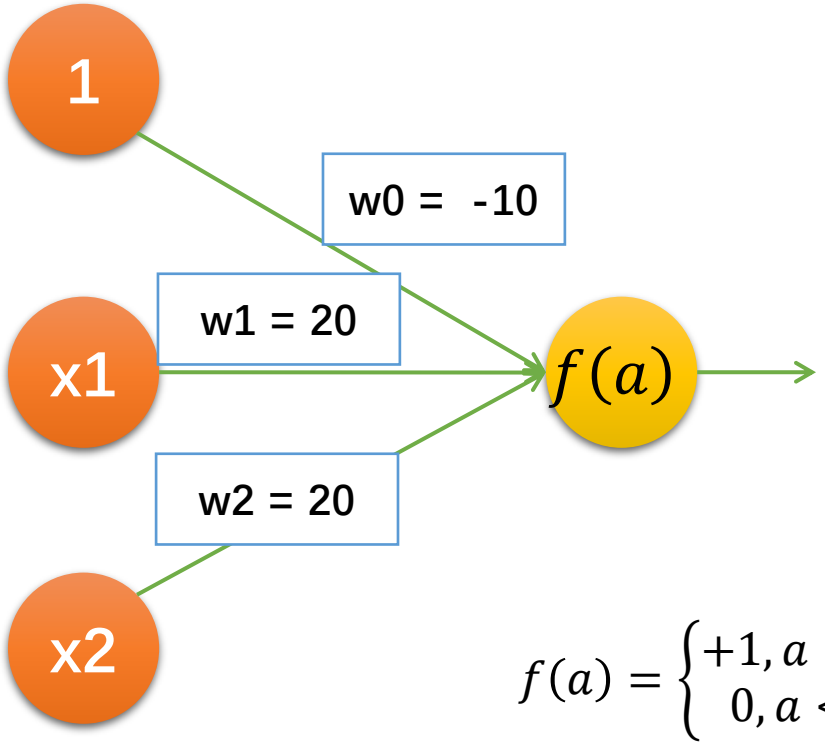
AND

x_1	x_2	t
0	0	0
0	1	0
1	0	0
1	1	1

XOR

x_1	x_2	t
0	0	1
0	1	0
1	0	0
1	1	1

感知机

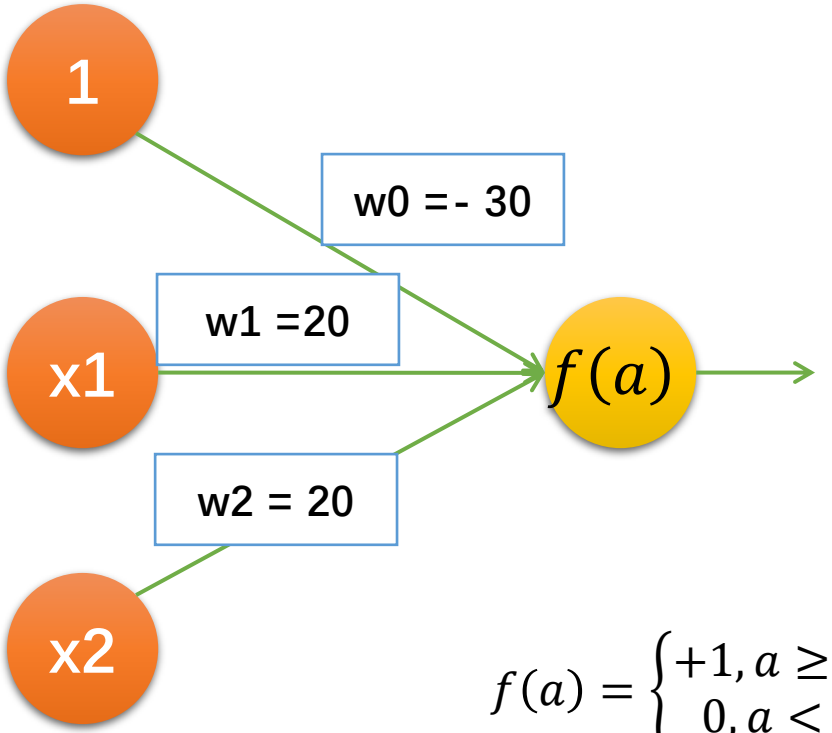


$$f(a) = \begin{cases} +1, a \geq 0 \\ 0, a < 0 \end{cases}$$

OR或

x_1	x_2	t	y
0	0	0	≈0
0	1	1	≈1
1	0	1	≈1
1	1	1	≈1

感知机

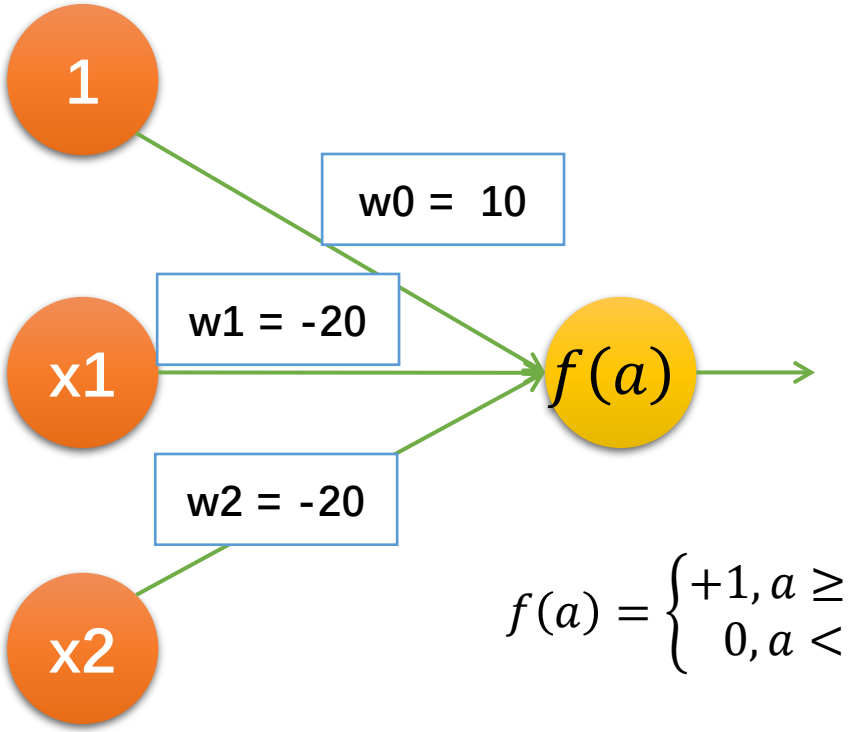


$$f(a)=\begin{cases}+1, a \geq 0\\ 0, a<0\end{cases}$$

AND与

x_1	x_2	t	y
0	0	0	≈0
0	1	0	≈0
1	0	0	≈0
1	1	1	≈1

感知机



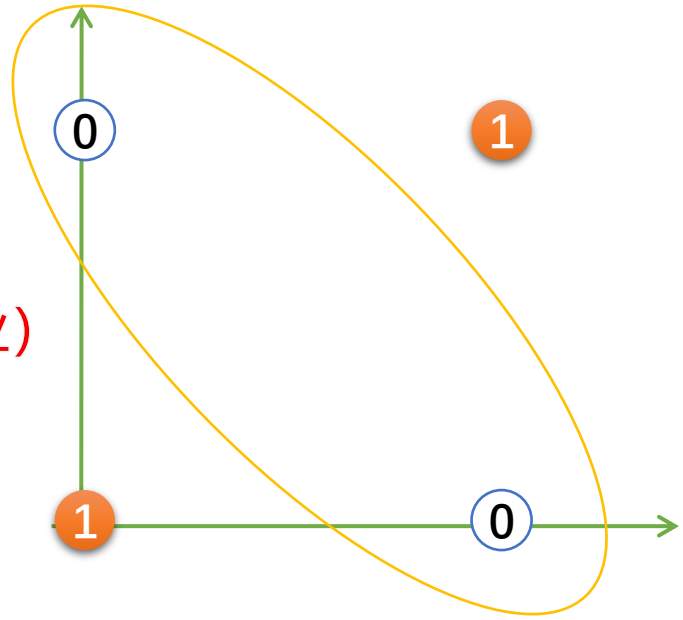
$$f(a) = \begin{cases} +1, a \geq 0 \\ 0, a < 0 \end{cases}$$

NOT X1 AND NOT X2

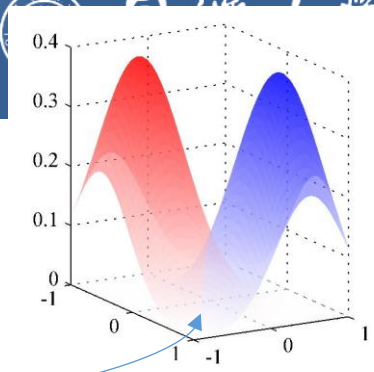
x_1	x_2	t	y
0	0	1	≈1
0	1	0	≈0
1	0	0	≈0
1	1	0	≈0

感知机

1. 无论如何调整参数，也无法解决线性不可分的XOR（异或）问题。
2. 只有决策边界为非线性的时候才可以解决此问题
3. 利用多层多个感知机(MLP，BP神经网络)
   - 多个感知机并联串联即可解决XOR问题，自行尝试（作业）
4. 利用核函数隐式学习非线性性质（SVM）
   - MLP的对偶问题，Memory-Based方法，自学



从概率与数理统计角度分析分类问题

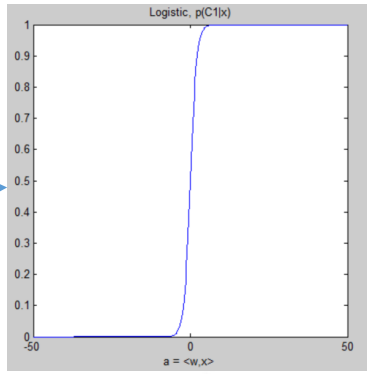


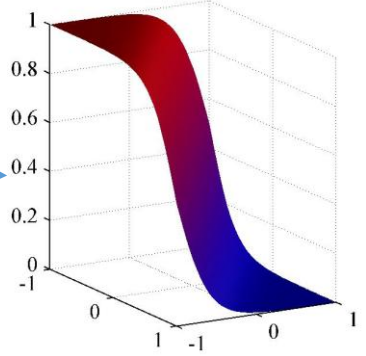
- 基本问题描述，首先考虑两类问题，给定x，判断类别C，$t=\begin{cases}1,\mathbf{x}\in C_1\\0,\mathbf{x}\in C_2\end{cases}$
  1. 类别后验概率$p(\mathbf{x}\epsilon C_k|\mathbf{x})$，记为$p(C_k|\mathbf{x})$，$p(C_1|\mathbf{x})=p(t=1|\mathbf{x})$，$p(C_2|\mathbf{x})=p(t=0|\mathbf{x})$
  2. 类别条件分布$p(\mathbf{x}|\mathbf{x}\epsilon C_k)$，记为$p(\mathbf{x}|C_k)$，一般可假设为正态分布$N(\boldsymbol{\mu}_k,\boldsymbol{\Sigma}_k)$
     - 如果**x**是离散变量，一般用多项分布或两项分布，自行学习
  3. 类别先验分布$p(\mathbf{x}\epsilon C_k)$记为$p(C_k)$，一般为伯努利分布$p(C_k)=\begin{cases}\pi,C_1\\1-\pi,C_2\end{cases}$，$p(t)=\pi^t(1-\pi)^{1-t}$
- 数据**x**的类别为C_1的后验概率

$$p(C_1|\mathbf{x})=p(t=1|\mathbf{x})=\frac{p(\mathbf{x}|C_1)p(C_1)}{p(\mathbf{x}|C_1)p(C_1)+p(\mathbf{x}|C_2)p(C_2)}=\frac{1}{1+\exp(-a)}=\sigma(a)$$

$$a=\ln\frac{p(\mathbf{x}|C_1)p(C_1)}{p(\mathbf{x}|C_2)p(C_2)}$$

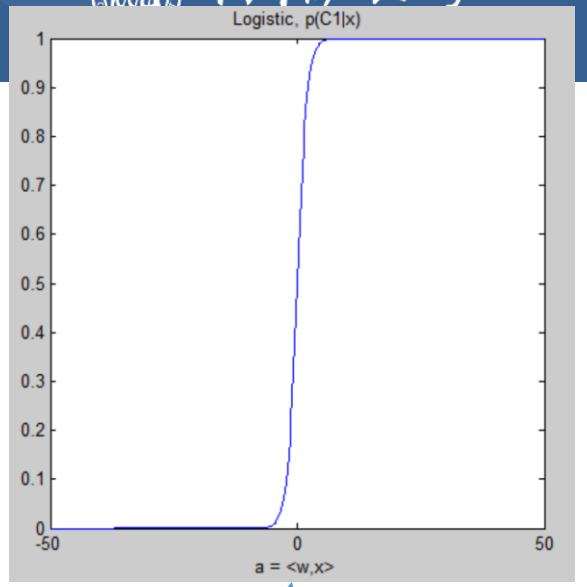
a的激活函数





- 如何求$p(C_1|\mathbf{x})$或$p(C_k|\mathbf{x})$是核心问题
  - 生成模型：对$p(t,\mathbf{x}|\boldsymbol{\theta})$进行参数估计，以参数$\boldsymbol{\theta}$(如$\pi,\boldsymbol{\mu},\boldsymbol{\Sigma}$)为自变量，从而求得$p(C_1|\mathbf{x})$，**自学**
  - 判别模型：以线性/非线性公式$f(\mathbf{w},\phi(\mathbf{x}))$对$a$近似，再对$p(t|\mathbf{w})$进行参数估计，从而得到$p(C_1|\mathbf{x})$

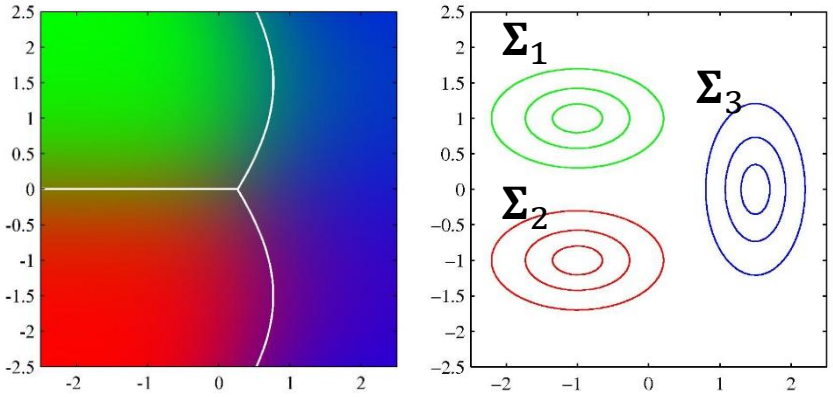
判别模型中对a的近似



- 数据x的类别为C_1的后验概率$p(C_1|\mathbf{x}) = \frac{1}{1+\exp(-a)} = \sigma(a),\ a = \ln\frac{p(\mathbf{x}|C_1)p(C_1)}{p(\mathbf{x}|C_2)p(C_2)}$
- 线性近似的适用范围：
  1. 后验概率 $p(C_k|\mathbf{x}) \propto p(\mathbf{x}|C_k)\,p(C_k) = p(\mathbf{x}, C_k)$
  2. 假设$\boldsymbol{\Sigma}_1 = \boldsymbol{\Sigma}_2 = \boldsymbol{\Sigma}$，将$p(\mathbf{x}|C_k) = N(\boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)$代入$p(C_1|\mathbf{x})$可得$a = \mathbf{w}^T\mathbf{x} + w_0$，线性模型

$$p(C_1|\mathbf{x}) = \sigma(\mathbf{w}^T\mathbf{x} + w_0), \begin{cases} \mathbf{w} = \boldsymbol{\Sigma}^{-1}(\boldsymbol{\mu}_1 - \boldsymbol{\mu}_2) \\ w_0 = -\frac{1}{2}\boldsymbol{\mu}_1{}^{\mathrm{T}}\boldsymbol{\Sigma}^{-1}\boldsymbol{\mu}_1 + \frac{1}{2}\boldsymbol{\mu}_2{}^{\mathrm{T}}\boldsymbol{\Sigma}^{-1}\boldsymbol{\mu}_2 + \ln\frac{p(C_1)}{p(C_2)} \end{cases}$$

  3. 分类面是直线或超平面$\sigma(a) = 0.5\ i.e.\ \mathbf{w}^T\mathbf{x} + w_0 = 0$
- 非线性，分类面$\sigma(a) = 0.5$不再对应线性模型$\mathbf{w}^T\mathbf{x} + w_0 = 0$
  - $\boldsymbol{\Sigma}_1 = \boldsymbol{\Sigma}_2 \neq \boldsymbol{\Sigma}_3$



判别模型与Logistic回归

- 后验概率是a被激活的结果， a可以用$f(\mathbf{w},\phi(\mathbf{x}))$表达，如果$f$是线性的，后验概率可写为：

$$p(C_1|\mathbf{x})=\sigma(a)=\sigma(\mathbf{w}^T\phi(\mathbf{x}))$$

- 单个数据的似然函数 $p(t\in\{0,1\}|\mathbf{w},\mathbf{x})=\big(p(t=1|\mathbf{x})\big)^t\big(p(t=0|\mathbf{x})\big)^{1-t}=p(C_1|\mathbf{x})^t\big(1-p(C_1|\mathbf{x})\big)^{1-t}$，$i.i.d$假设，可得N个数据的似然函数：

$$p(\mathbf{t}|\mathbf{w})=\prod_{n=1}^{N}p(C_1|\mathbf{x}_n)^{t_n}\big(1-p(C_1|\mathbf{x}_n)\big)^{1-t_n}=\prod_{n=1}^{N}\sigma(a_n)^{t_n}\big(1-\sigma(a_n)\big)^{1-t_n}$$

$$\mathrm{Loss}(\mathbf{w})=-\ln p(\mathbf{t}|\mathbf{w})=-\sum_{n=1}^{N}\{t_n\ln\sigma(a_n)+(1-t_n)\ln\big(1-\sigma(a_n)\big)\}$$

两类问题的交叉熵损失函数

$$a=\mathbf{w}^T\phi(\mathbf{x})$$

$$\frac{d\sigma}{da}=\sigma(1-\sigma)$$

$$\frac{da}{d\mathbf{w}}=\phi(\mathbf{x})$$

$$\nabla\mathrm{Loss}(\mathbf{w})=-\sum_{n=1}^{N}\left\{t_n\frac{1}{\sigma_n}\sigma_n(1-\sigma_n)\phi(\mathbf{x}_n)+(1-t_n)\frac{1}{1-\sigma_n}\big(-\sigma_n(1-\sigma_n)\big)\boldsymbol{\phi}(\mathbf{x}_n)\right\}$$

$$=-\sum_{n=1}^{N}\{t_n(1-\sigma_n)\boldsymbol{\phi}(\mathbf{x})-(1-t_n)\sigma_n\boldsymbol{\phi}(\mathbf{x}_n)\}=-\sum_{n=1}^{N}(t_n-\sigma_nt_n-\sigma_n+\sigma_nt_n)\boldsymbol{\phi}(\mathbf{x}_n)=\sum_{n=1}^{N}(\sigma_n-t_n)\boldsymbol{\phi}(\mathbf{x}_n)$$

Logistic回归与多项式回归的似然函数对比

$$p(C_1|\mathbf{x}) = \sigma(a) = \sigma\left(\mathbf{w}^T\phi(\mathbf{x})\right) \qquad a = \ln\frac{p(\mathbf{x}|C_1)p(C_1)}{p(\mathbf{x}|C_2)p(C_2)}$$

- 分类问题：观测值t的似然度$p(t \in \{0,1\}|\mathbf{w},\mathbf{x}) = \left(p(t=1|\mathbf{w},\mathbf{x})\right)^t\left(p(t=0|\mathbf{w},\mathbf{x})\right)^{1-t} = p(C_1|\mathbf{x})^t\left(1-p(C_1|\mathbf{x})\right)^{1-t}$

$$= \left(\sigma(a)\right)^t\left(1-\sigma(a)\right)^{1-t} = \left(\sigma\left(\mathbf{w}^T\phi(\mathbf{x})\right)\right)^t\left(1-\sigma\left(\mathbf{w}^T\phi(\mathbf{x})\right)\right)^{1-t}$$

- N个$i.i.d$数据$\mathbf{t} = (t_1, t_2, \dots, t_N)^T$的联合似然：$p(\mathbf{t}|\mathbf{w}) = \prod_{n=1}^N \sigma(a_n)^{t_n}\left(1-\sigma(a_n)\right)^{1-t_n}$
- 自然对数似然函数为

$$\ln p(\mathbf{t}|\mathbf{w}) = \sum_{n=1}^N \left\{t_n \ln\sigma(a_n) + (1-t_n)\ln\left(1-\sigma(a_n)\right)\right\}$$

- 回归问题：观测值t的似然度$p(t \in R|\mathbf{w},\mathbf{x}) = \mathcal{N}(t|\mathbf{w}^T\phi(\mathbf{x}),\sigma^2)$
- N个$i.i.d$数据$\mathbf{t} = (t_1, t_2, \dots, t_N)^T$的联合似然$p(\mathbf{t}|\mathbf{w}) = \prod_{n=1}^N \mathcal{N}(t_n|\mathbf{w}^T\phi(\mathbf{x}_n),\sigma^2)$
- 自然对数似然函数为

$$\ln p(\mathbf{t}|\mathbf{w},\sigma^2) = -\frac{1}{2\sigma^2}\sum_{n=1}^N (t_n - \mathbf{w}^T\mathbf{x}_n)^2 - \frac{N}{2}\ln\sigma^2 - \frac{N}{2}\ln(2\pi)$$

判别模型 Logistic回归的SGD求解

$$\text{Loss}(\mathbf{w}) = -\ln p(\mathbf{y}|\mathbf{w}) = -\sum_{n=1}^{N}\{t_n \ln\sigma(a_n) + (1-t_n)\ln(1-\sigma(a_n))\}$$

- 是否有闭合（Closed-Form Solution）解?

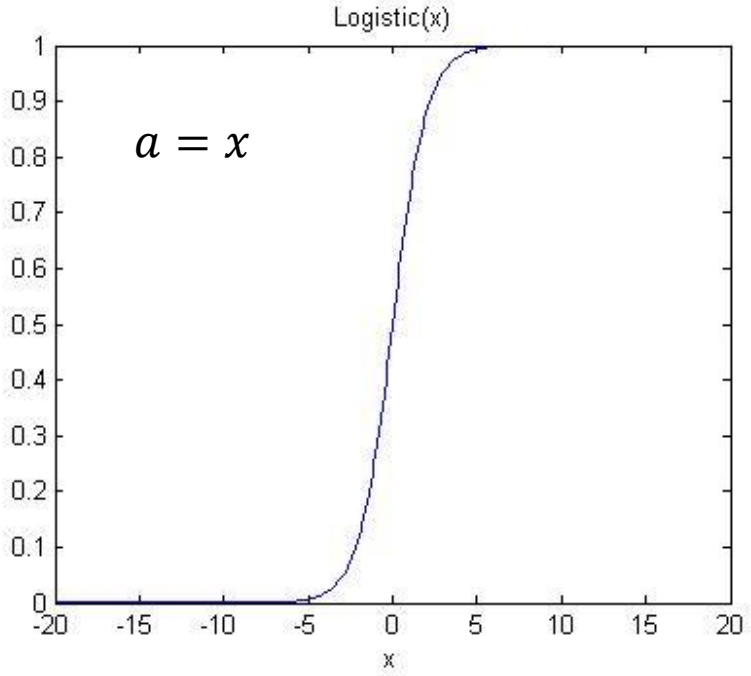
$$\sigma(a) = \frac{1}{1+\exp(-a)}, a = \mathbf{w}^T\boldsymbol{\phi}(\mathbf{x})$$

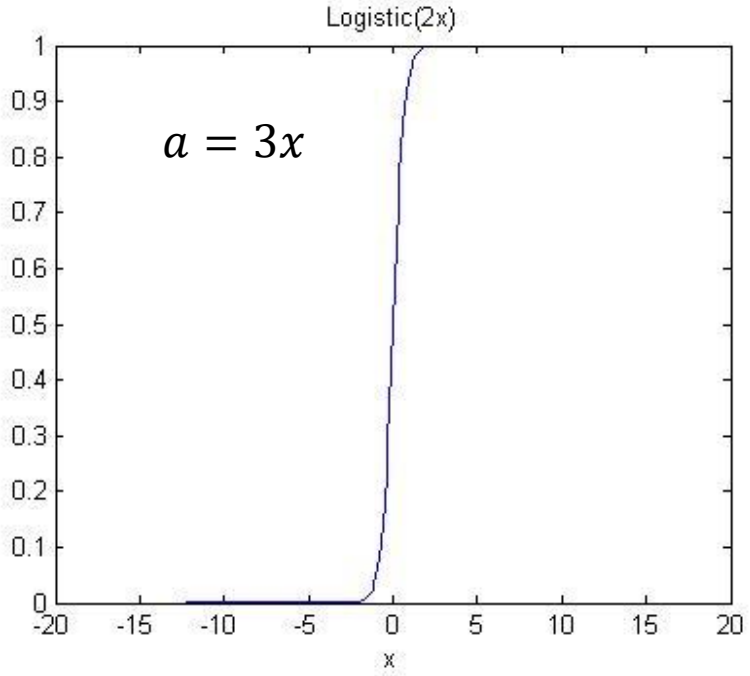
$$\nabla\text{Loss}(\mathbf{w}) = \sum_{n=1}^{N}(\sigma_n - t_n)\boldsymbol{\phi}(\mathbf{x}_n) = 0$$

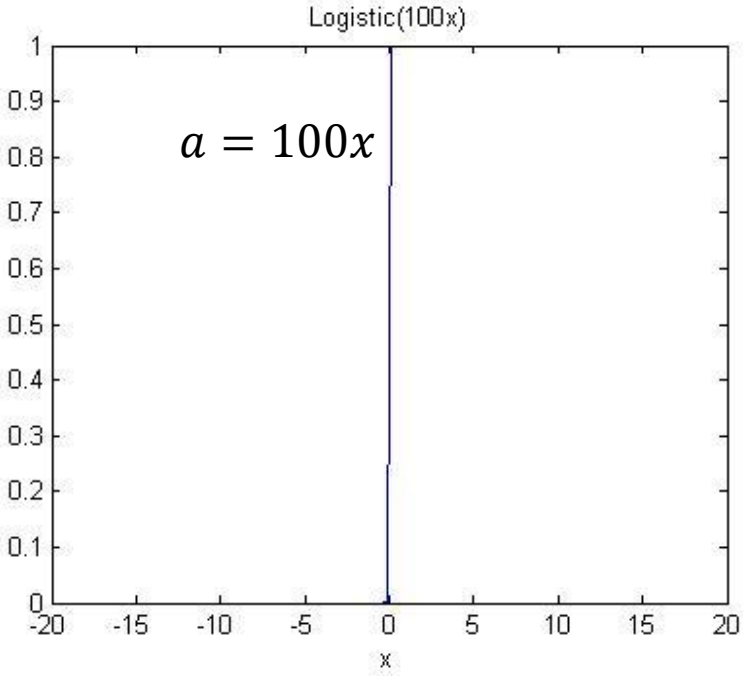
- 没有，因为在$\nabla\text{Loss}(\mathbf{w}) = 0$中，w包含在$\sigma$中，$\sigma$不是w的线性函数，无法得到线性方程组。
- 基于一阶导数的搜索方法（Stochastic Gradient Descent, SGD）
- 基于二阶导数的搜索方法（牛顿法，Newton-Raphson，Iterative Reweighted Least Squares, IRLS）

判别模型 Logistic回归的过拟合问题

- 过拟合，陡峭， w趋于无穷大，此时损失最小 $\mathbf{w}^T\phi(\mathbf{x})$
- 类似贝叶斯回归，将w的范数（w的先验概率）作为正则化项，加入优化问题







多类 Logistic回归

$$p(C_k|\mathbf{x}) = \text{softmax}(a) = \frac{\exp(a_k)}{\sum_j \exp(a_j)}, a_k = \mathbf{w}_k{}^{\mathrm{T}}\phi(\mathbf{x})$$

$$\mathbf{w}_k = (w_0, w_1, w_2, \cdots, w_M)^{\mathrm{T}}$$

两类问题是多类问题的特例

$$p(C_1|\mathbf{x}) = \sigma(a) = \frac{1}{1+\exp(-a)} = \frac{\exp(a)}{\exp(a)+1}, a = \mathbf{w}^T\phi(\mathbf{x})$$

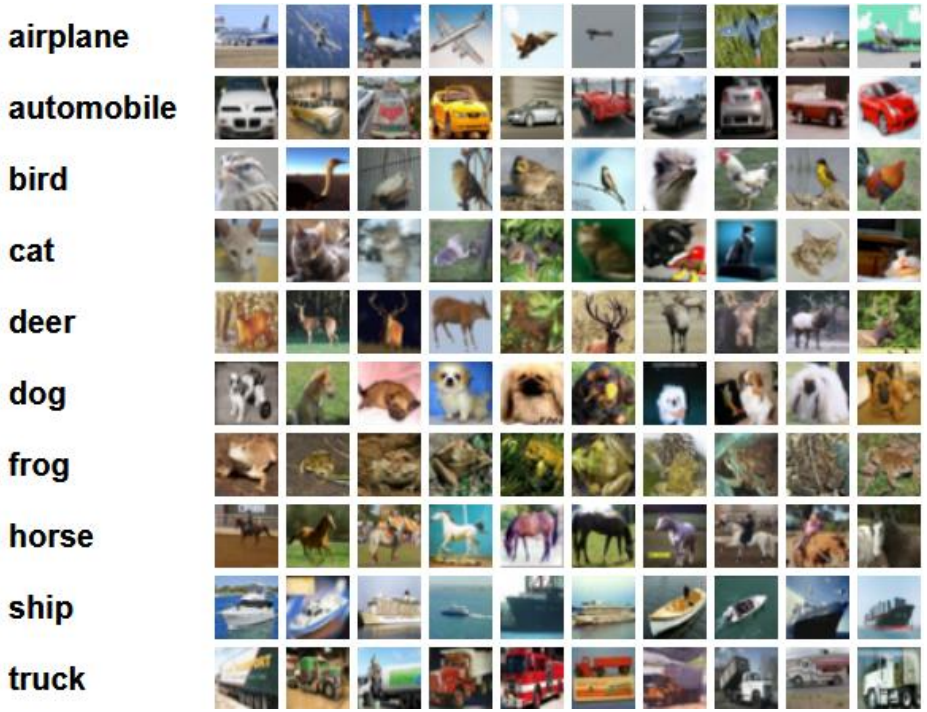
$$\mathbf{w}_1 = (w_0, w_1, w_2, \cdots, w_M)^{\mathrm{T}} \quad a_1 = \mathbf{w}_1{}^T\phi(\mathbf{x})$$

$$p(C_1|\mathbf{x}) = \frac{\exp(a_1)}{\exp(a_1)+\exp(a_2)}$$

$$\mathbf{w}_2 = (0,0,0,\cdots,0)^{\mathrm{T}} \quad a_2 = \mathbf{w}_2{}^T\phi(\mathbf{x}) = 0$$

$$p(C_2|\mathbf{x}) = 1 - \sigma(a_1) = \frac{\exp(a_2)}{\exp(a_1)+\exp(a_2)}$$

作业：CIFAR-10等



CIFAR-10，10个类别，每个类别60000图像
32×32，50000训练，10000测试

多类 Logistic回归

$$p(C_k|\mathbf{x}) = \text{softmax}(a_k) = \frac{\exp(a_k)}{\sum_j \exp(a_j)}, a_k = \mathbf{w}_k{}^{\mathrm{T}}\phi(\mathbf{x}), \qquad \mathbf{w}_k = (w_0, w_1, w_2, \cdots, w_M)^{\mathrm{T}}$$

两类问题是多类问题的特例

$$p(\mathbf{t}|\mathbf{w}) = \prod_{n=1}^{N} p(C_1|\mathbf{x}_n)^{t_n}\left(1 - p(C_1|\mathbf{x}_n)\right)^{1-t_n} = \prod_{n=1}^{N} \sigma(a_n)^{t_n}\left(1 - \sigma(a_n)\right)^{1-t_n}$$

$$p(\mathbf{T}|\mathbf{w}_1, \mathbf{w}_2, \cdots, \mathbf{w}_K) = \prod_{n=1}^{N}\prod_{k=1}^{K} p(C_k|\phi(\mathbf{x}_n))^{t_{nk}} = \prod_{n=1}^{N}\prod_{k=1}^{K} \text{softmax}(a_{nk})^{t_{nk}}, a_{nk} = \mathbf{w}_k{}^{\mathrm{T}}\phi(\mathbf{x}_n)$$

$$\text{Loss}(\mathbf{w}_1, \mathbf{w}_2, \cdots, \mathbf{w}_K) = -\ln p(\mathbf{T}|\mathbf{w}_1, \mathbf{w}_2, \cdots, \mathbf{w}_K) = -\sum_{n=1}^{N}\sum_{k=1}^{K} t_{nk} \ln\{\text{softmax}(a_{nk})\}$$

$$\frac{\partial \text{softmax}(a_k)}{\partial a_j} = \text{softmax}(a_k)\left(I_{kj} - \text{softmax}(a_j)\right) \qquad I_{kj} = \begin{cases} 1, k = j \\ 0, k \neq j \end{cases}$$

$$\mathbf{T} = \begin{pmatrix} \mathbf{t}_1^T \\ \mathbf{t}_2^T \\ \vdots \\ \mathbf{t}_N^T \end{pmatrix} = \begin{pmatrix} t_{11} & t_{12} & \cdots & t_{1K} \\ t_{21} & t_{22} & \cdots & t_{2K} \\ \vdots & \vdots & \ddots & \vdots \\ t_{N1} & t_{N2} & \cdots & t_{NK} \end{pmatrix}$$

多类 Logistic回归

多类问题的交叉熵损失函数

$$\mathrm{Loss}(\mathbf{w}_1,\mathbf{w}_2,\cdots,\mathbf{w}_K)=-\ln p(\mathbf{T}|\mathbf{w}_1,\mathbf{w}_2,\cdots,\mathbf{w}_K)=-\sum_{n=1}^{N}\sum_{k=1}^{K}t_{nk}\ln\{\mathrm{softmax}(a_{nk})\}$$

$$\sum_{k=1}^{K}t_k=1$$

$$\frac{\partial\mathrm{Loss}(\mathbf{w}_1,\mathbf{w}_2,\cdots,\mathbf{w}_K)}{\partial\mathbf{w}_j}=-\sum_{n=1}^{N}\sum_{k=1}^{K}t_{nk}\frac{1}{\mathrm{softmax}(a_{nk})}\left\{\mathrm{softmax}(a_{nk})\left(I_{kj}-\mathrm{softmax}(a_{nj})\right)\right\}\phi(\mathbf{x}_n)$$

$$=-\sum_{n=1}^{N}\sum_{k=1}^{K}t_{nk}\left(I_{kj}-\mathrm{softmax}(a_{nj})\right)\phi(\mathbf{x}_n)=-\sum_{n=1}^{N}\left\{t_{nj}\left(1-\mathrm{softmax}(a_{nj})\right)+\sum_{k\neq j}^{K}-t_{nk}\mathrm{softmax}(a_{nj})\right\}\phi(\mathbf{x}_n)$$

$$=-\sum_{n=1}^{N}\left\{t_{nj}-t_{nj}\mathrm{softmax}(a_{nj})-\mathrm{softmax}(a_{nj})+\mathrm{softmax}(a_{nj})t_{nj}\right\}\phi(\mathbf{x}_n)=\sum_{n=1}^{N}\left\{\mathrm{softmax}(a_{nj})-t_{nj}\right\}\phi(\mathbf{x}_n)$$

$$\sum_{k\neq j}^{K}t_{nk}\mathrm{softmax}(a_{nj})=\mathrm{softmax}(a_{nj})\sum_{k\neq j}^{K}t_{nk}=\mathrm{softmax}(a_{nj})(1-t_{nj})$$

$$\frac{\partial\mathrm{softmax}(a_k)}{\partial a_j}=\mathrm{softmax}(a_k)\left(I_{kj}-\mathrm{softmax}(a_j)\right)\qquad I_{kj}=\begin{cases}1,k=j\\0,k\neq j\end{cases}\qquad a_k=\mathbf{w}_k{}^{\mathrm{T}}\phi(\mathbf{x})$$

多类 Logistic回归

$$p(C_k|\mathbf{x}) = \text{softmax}(a) = \frac{\exp(a_k)}{\sum_j \exp(a_j)}, a_k = \mathbf{w}_k{}^{\mathrm{T}}\phi(\mathbf{x})$$

$$\text{Loss}(\mathbf{w}_1, \mathbf{w}_2, \cdots, \mathbf{w}_K) = -\ln p(\mathbf{T}|\mathbf{w}_1, \mathbf{w}_2, \cdots, \mathbf{w}_K) = -\sum_{n=1}^{N}\sum_{k=1}^{K} t_{nk}\ln\{\text{softmax}(a_{nk})\}$$

SGD求解（一阶导数），第j类的权值$\mathbf{w}_j$的导数：

$$\nabla_{\mathbf{w}_j}\text{Loss}(\mathbf{w}_1, \mathbf{w}_2, \cdots, \mathbf{w}_K) = \sum_{n=1}^{N}\{\underbrace{\text{softmax}(a_{nj})}_{\text{预测值}} - \underbrace{t_{nj}}_{\text{观测值}}\}\phi(\mathbf{x}_n) = \sum_{n=1}^{N}\{p(C_j|\mathbf{x}_n) - t_{nj}\}\phi(\mathbf{x}_n)$$

预测值：属于第j类的后验概率

观测值：属于第j类的标签

例：$j = 2$

$y = (0.1, \quad 0.8, \quad 0.05, \quad 0.025, \quad 0.025)^T$

$y = (0.1, \quad 0.1, \quad 0.10, \quad 0.790, \quad 0.010)^T$

$\mathrm{t} = (0, \quad 1, \quad 0, \quad 0, \quad 0)^T$

Logistic回归与感知机

1. 感知机的损失函数：$E_p(\mathbf{w}) = -\sum_{n\in\mathcal{M}} \mathbf{w}^T\boldsymbol{\phi}(\mathbf{x}_n)t_n = -\sum_{n=1}^{N}\frac{(y_n-t_n)}{2}\mathbf{w}^T\boldsymbol{\phi}(\mathbf{x}_n)t_n$

   - 单个数据的损失$E_p(\mathbf{w})_n = -\sum_{n\in\mathcal{M}} \mathbf{w}^T\boldsymbol{\phi}(\mathbf{x}_n)t_n = \begin{cases} -\mathbf{w}^T\boldsymbol{\phi}(\mathbf{x}_n), t_n = 1, y_n = -1 \\ \mathbf{w}^T\boldsymbol{\phi}(\mathbf{x}_n), t_n = -1, y_n = 1 \\ 0 \quad , \quad else \end{cases}$

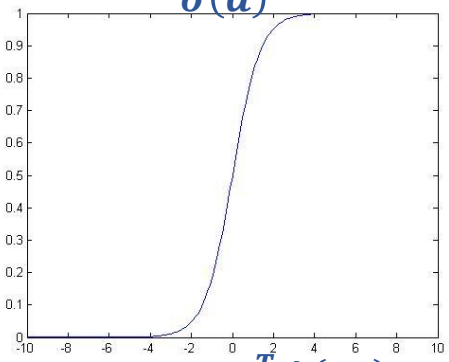
2. 两类Logistic回归损失函数$\mathrm{Loss}(\mathbf{w}) = -\sum_{n=1}^{N}\{t_n \ln\sigma(a_n) + (1-t_n)\ln(1-\sigma(a_n))\}, a_n = \mathbf{w}^T\phi(\mathbf{x}_n)$

   - 单个数据的损失 $\mathrm{Loss}(\mathbf{w})_n = \begin{cases} -\ln\sigma(a_n), t_n = 1, y_n = \sigma_n \approx 0 \; or \; \sigma_n < 0.5 \\ -\ln(1-\sigma(a_n)), t_n = 0, y_n = \sigma_n \approx 1 \; or \; \sigma_n \geq 0.5 \\ 0 \quad , \quad else \end{cases}$

特征的线性函数以激活函数的对数形式体现在损失中

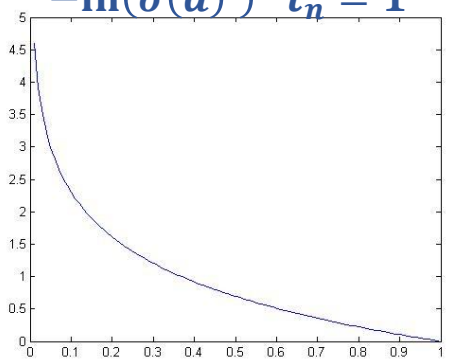
- 非线性优化问题
- 但目标函数为凸

$\sigma(a)$



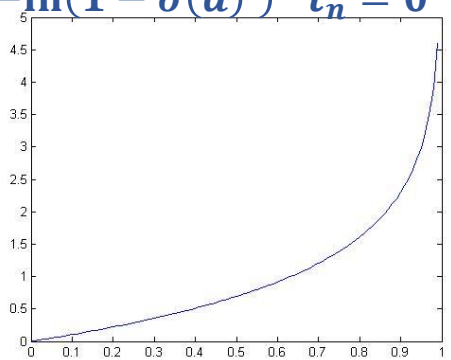
$a = \mathbf{w}^T\boldsymbol{\phi}(\mathbf{x}_n)$

$-\ln(\sigma(a))\quad t_n = 1$



$\sigma(a)$

$-\ln(1-\sigma(a))\quad t_n = 0$



$\sigma(a)$

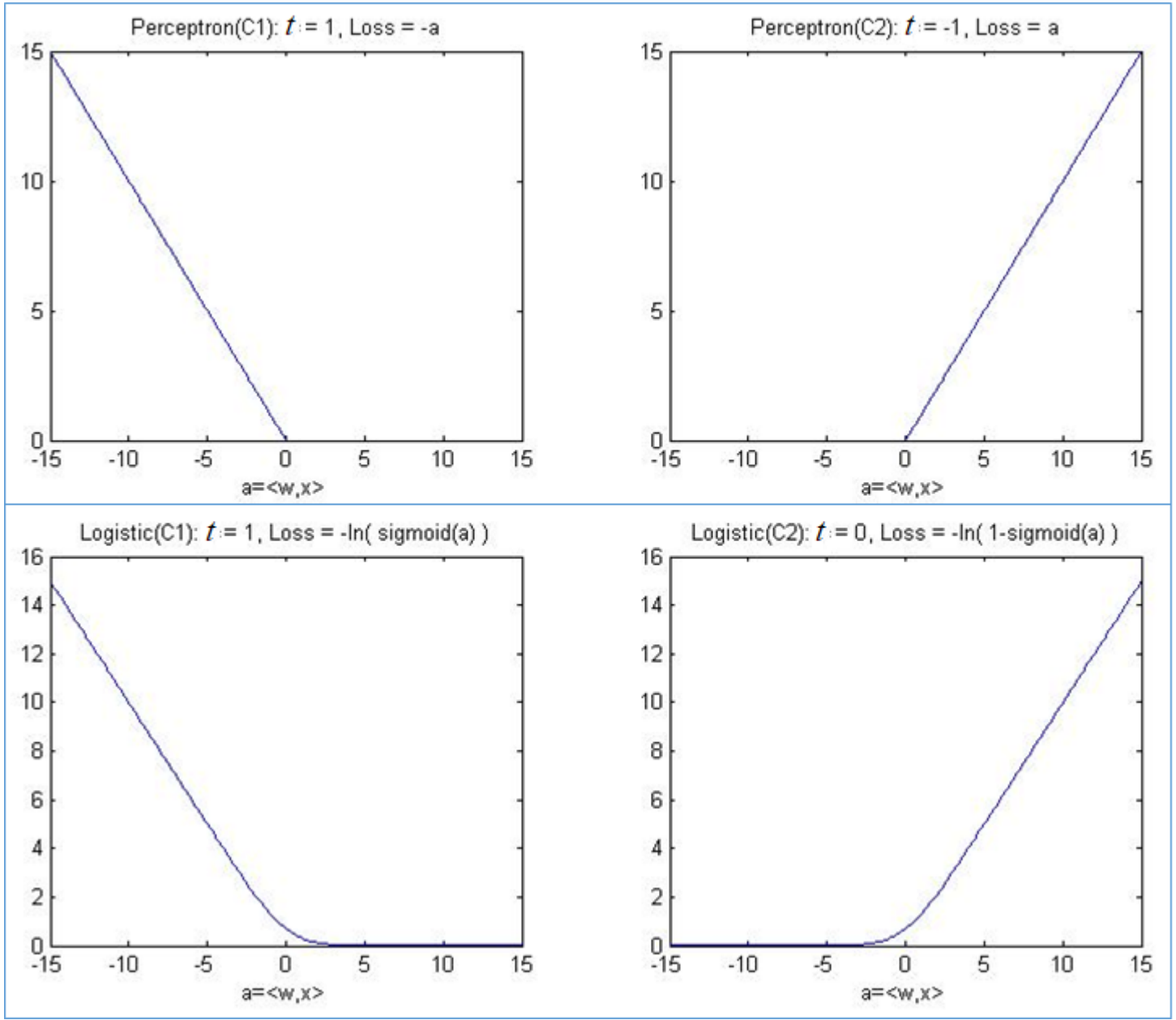
交叉熵损失函数由两部分构成（y=1,y=0）两个部分对w而言都是光滑（凸Convex ）的，没有局部极小值。

Logistic回归与感知机

$$E_p(\mathbf{w})_n = \begin{cases} -\mathbf{w}^T\boldsymbol{\phi}(\mathbf{x}_n), t_n = 1, y_n = -1 \\ \mathbf{w}^T\boldsymbol{\phi}(\mathbf{x}_n), t_n = -1, y_n = 1 \\ 0 \quad , \quad else \end{cases}$$

- 特征的线性函数为自变量 $a_n = \mathbf{w}^T\phi(\mathbf{x}_n)$
- 查看损失函数形态，两者基本一致。
- Logistic回归是全程可导的平滑版本。

$$\mathrm{Loss}(\mathbf{w})_n = \begin{cases} -\ln\sigma(a_n), t_n = 1, y_n = \sigma_n \approx 0 \; or \; a_n < 0 \\ -\ln\big(1-\sigma(a_n)\big), t_n = 0, y_n = \sigma_n \approx 1 \; or \; a_n \geq 0 \\ 0 \quad , \quad else \end{cases}$$



Logistic回归与感知机

1. 感知机权值更新：$\nabla E_p(\mathbf{w}) = -\sum_{n\in\mathcal{M}} \boldsymbol{\phi}(\mathbf{x}_n)t_n = \sum_{n=1}^{N} \frac{(y_n - t_n)}{2}\boldsymbol{\phi}(\mathbf{x}_n)t_n$

   - 单个数据的损失的导数 $\nabla E_p(\mathbf{w})_n = \begin{cases} -\boldsymbol{\phi}(\mathbf{x}_n), t_n = 1 \ , y_n = -1 \\ \boldsymbol{\phi}(\mathbf{x}_n), t_n = -1, y_n = 1 \\ 0 \quad , \qquad else \end{cases}$ 只对错分数据计算

2. 两类Logistic回归权值更新：$\nabla \mathrm{Loss}(\mathbf{w}) = \sum_{n=1}^{N}(\sigma_n - t_n)\boldsymbol{\phi}(\mathbf{x}_n)$

   - 单个数据的损失的导数 $\nabla \mathrm{Loss}(\mathbf{w})_n \approx \begin{cases} (\sigma_n - 1)\boldsymbol{\phi}(\mathbf{x}_n), t_n = 1, y_n = \sigma_n \approx 0 \ or \ \sigma_n < 0.5 \\ \sigma_n\boldsymbol{\phi}(\mathbf{x}_n), t_n = 0, y_n = \sigma_n \approx 1 \ or \ \sigma_n \geq 0.5 \\ 0 \quad , \qquad else \end{cases}$ 对所有数据都可计算

Logistic回归与线性回归的对比

- Logistic回归，

$$a = \mathbf{w}^{\mathrm{T}}\phi(\mathbf{x}_n)$$

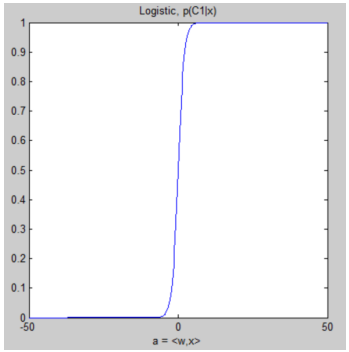
$$\mathrm{Loss}(\mathbf{w}) = -\sum_{n=1}^{N}\{t_n \ln\sigma(a_n) + (1-t_n)\ln(1-\sigma(a_n))\}$$

$$\nabla\mathrm{Loss}(\mathbf{w}) = \sum_{n=1}^{N}(\sigma_n - t_n)\boldsymbol{\phi}(\mathbf{x}_n) = \boldsymbol{\Phi}^T(\boldsymbol{\sigma} - \mathbf{t})$$

- 线性回归，

$$Loss(\mathbf{w}) = \frac{1}{2}\|\boldsymbol{\Phi}\mathbf{w} - \mathbf{t}\|^2 = \frac{1}{2}\sum_{n=1}^{N}(\mathbf{w}^T\boldsymbol{\phi}(\mathbf{x}_n) - t_n)^2$$

$$\nabla Loss(\mathbf{w}) = \boldsymbol{\Phi}^T(\boldsymbol{\Phi}\mathbf{w} - \mathbf{t})$$

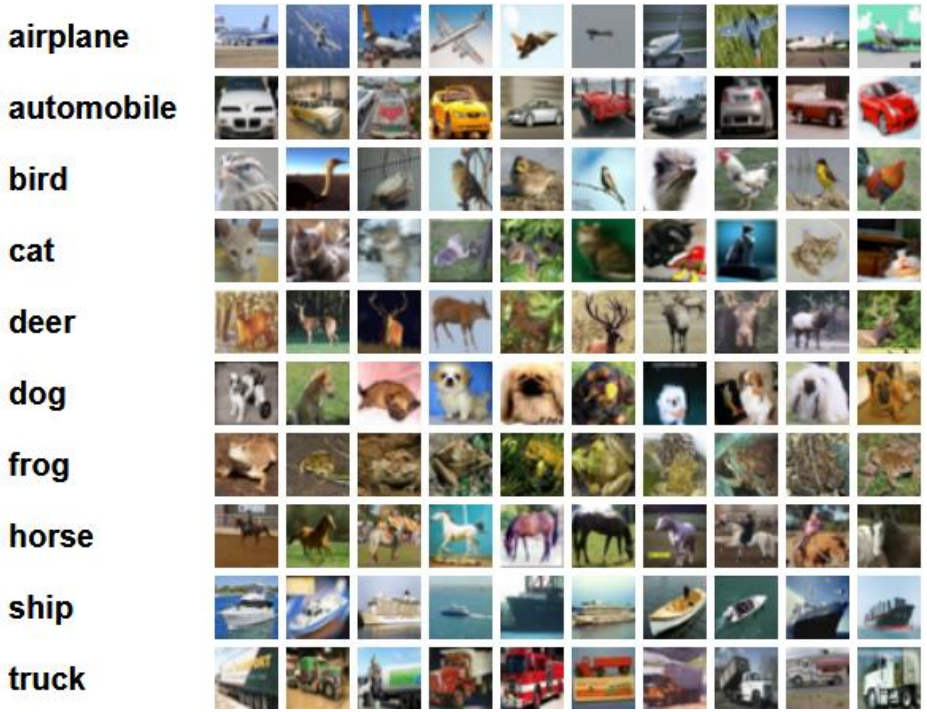


1. 预测值：
   1. Logistic回归用线性函数的激活输出作为预测值
   2. 线性回归用线性函数的值作为预测值
2. 损失函数：
   1. Logistic回归的损失函数为离散分布导出的对数似然函数
   2. 线性回归损失函数为正态分布导出的对数似然函数
3. 求解：
   1. Logistic回归只能迭代求解
   2. 线性回归可直接求解

作业

Logistic回归，图像原始像素作为特征。

CIFAR-10，分类



感谢！



总结

- Linear Discriminant Functions have simple geometry
- Extensible to multiple classes
- Parameters can be learnt using
  - Least squares
    - not robust to outliers, model close to target values
  - Fisher’ s linear discriminant
    - Two class is special case of least squares
    - Not easily generalized to more classes
  - Perceptrons
    - Does not converge if classes not linearly separable
    - Does not provide probabilistic output
    - Not readily generalized to K>2 classes